

MGQL: An Executable, Small-Step Semantics of GQL

ADITYA THIMMAIAH, University of Texas at Austin, USA

TONG-NONG LIN, University of Texas at Austin, USA

MILOS GLIGORIC, University of Texas at Austin, USA

Research and development of graph query languages has been gaining traction with the increase in popularity of graph databases, specifically due to the flexible schema and other rich semantic offerings of the latter's most common underlying data model: the property graph. This has culminated in the standardization of the ISO Graph Query Language (GQL) as ISO/IEC 39075 in 2024, the first international standard for property graph-based graph query languages. However, ISO/IEC 39075 codifies its semantics informally across 600+ pages of prose, making it difficult to formally reason about the standard or for a standard-faithful implementation.

Existing formalizations are not adequate because they either: (1) significantly reduce the semantic complexity by omitting bag semantics, schemas, and composite queries on multiple graphs; (2) or significantly reduce the syntactic complexity by only considering isolated fragments such as pattern-matching, leaving the full query pipeline unformalized. Yet it is these semantic-syntactic features that make formalizing GQL non-trivial.

We present MGQL, the first mechanized, small-step operational semantics for a substantial read-only fragment of GQL that is grounded in the ISO/IEC 39075 standard. Our formalization models multi-graph property graphs with mixed edge directionality and supports a large fraction of GQL pattern constructs: quantified paths and edges, directional and undirected matching, label expressions, pattern lists, and composite queries. The semantics is supported by a schema-aware type system that refines variable types via closed-graph schemas, tracks nullability, supports multiple composite query operators, and models quantified-path bindings with list types. We prove that our type system is sound, ensuring an end-to-end guarantee of well-formed queries yielding results that conform to their declared schemas. MGQL provides the first bridge between GQL's informal specification and a mechanized implementation, enabling formal reasoning about correctness.

CCS Concepts: • **Theory of computation** → **Type theory; Operational semantics**; • **Software and its engineering** → **Semantics**; • **Information systems** → **Query languages for non-relational engines**.

Additional Key Words and Phrases: ISO GQL, Cypher, Graph Query Language, Formal Semantics, Lean, Mechanization

ACM Reference Format:

Aditya Thimmaiah, Tong-Nong Lin, and Milos Gligoric. 2026. MGQL: An Executable, Small-Step Semantics of GQL. *Proc. ACM Program. Lang.* 10, OOPSLA2, Article 327 (October 2026), 27 pages. <https://doi.org/10.1145/3839459>

1 Introduction

Graph databases have become foundational for modern data management and power many applications from social networks and fraud detection to drug discovery and knowledge graphs [1, 27]. The Graph Query Language (GQL) [11, 16], standardized in 2024 as ISO/IEC 39075, is the first international standard of graph query languages that query property graphs [1]. The standard

Authors' Contact Information: Aditya Thimmaiah, University of Texas at Austin, Austin, USA, auditt@utexas.edu; Tong-Nong Lin, University of Texas at Austin, Austin, USA, tong-nong@utexas.edu; Milos Gligoric, University of Texas at Austin, Austin, USA, gligoric@utexas.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2475-1421/2026/10-ART327

<https://doi.org/10.1145/3839459>

unified language features from other graph query languages such as Cypher [15], PGQL [30], and G-CORE [2]. It defines the semantics specification which implementations are expected to conform.

However, the semantics is specified informally, spanning more than 600 pages of prose that interleave data flow, typing, query evaluation pipelines, and pattern-matching, etc., making formally reasoning about GQL, such as verifying implementations or proving query equivalences, difficult.

Existing formalizations do not address this adequately. Francis et al. [13] distilled GQL’s pattern matching into a Graph Pattern Calculus (GPC) with typing rules and a denotational semantics, and Gheerbrant et al. [16] defined Core GQL and Core PGQ to study its expressive power, but both leave out graph schemas [4] and bag semantics. Ye et al. [31] formalized a gradually typed calculus for GQL path patterns in isolation from the rest of the language, implementing it in Python. No mechanized semantics exists for a fragment of GQL rich enough to model the interaction between graph schemas, bag semantics, and composite queries, etc., some of the key unique features of GQL.

We present MGQL, the first mechanized, executable, small-step operational semantics for a large read-only fragment of GQL, together with a schema-aware type system and a type soundness theorem, all machine-checked in Lean4 [9]; providing a foundation for rigorous formal reasoning.

We handle the complexity of GQL’s semantics by *stratifying* our static typing rules and small-step operational semantics into compositional layers—expressions, patterns, and queries. This allows us to capture the key non-trivial features of GQL. First, graph element types in GQL are graph-scoped and refined by graph schemas, allowing for label and property constraints to enable statically detecting patterns that cannot match, but requiring a typing system that propagates refined types across the query pipeline. Second, GQL’s three-valued semantics identifies null with the truth value *Unknown*, requiring every operator to participate in Kleene-style null propagation and every type to track nullability. Third, GQL’s *path modes* and *match modes* impose additional constraints on pattern matching such as the TRAIL path mode which constrains a pattern-match by forbidding repeated edges, thus breaking naïve path composition. We handle this by introducing auxiliary *execution constructs* besides those specified in the standard, which carry the edges visited during pattern-matching alongside the matched elements, making it trivial to enforce edge-disjointness. Fourth, quantified path patterns iterate over graph structure and bind their variables to sequences rather than single elements. We compute their repetitions with a frontier. Finally, composite queries combine bags whose records carry the same attributes but not necessarily the same types, yielding heterogeneous unions that the type system must track to formalize well-formedness of queries.

We make the following contributions:

- We define a **formal calculus** (§2.2) for a rich read-only fragment of GQL, covering graph resolution, pattern matching, filtering, projection, and composite queries over multiple graphs.
- We develop the first **schema-aware type system** (§3) for GQL, supporting graph-scoped refinement, nullability tracking, heterogeneous unions, and list-typed quantified path bindings.
- We give the first **small-step operational semantics** (§5) for GQL, capturing TRAIL-aware pattern matching, quantified patterns, nulls, and composite queries; all under bag semantics.
- We prove a **layered type soundness theorem** (§6) guaranteeing that well-formed queries (on complete evaluation) produce results that conform to the types assigned to them statically.
- We **mechanize** (§7) our semantics in Lean4, and validate it on queries of the Linked Data Benchmark Council’s (LDBC) Social Network Benchmark (SNB) Interactive v2 workload [22].

All references to the GQL standard are with respect to the ISO/IEC 39075 First Edition 2024, which we henceforth refer to as just **ISO**. We also abbreviate normative *syntax rules* from the standard as **SR** when referring to them. MGQL is available at <https://github.com/EngineeringSoftware/mgql>.

2 Preliminaries

We now formally introduce the two core components our work is built around: the property graph data model (§2.1) and the GQL graph query language (§2.2) that operates on them.

Notation 2.1 (Metafunctions and Shorthands). We use the metafunctions **dom**(·) and **supp**(·) to denote the *domain* and the *support* of a function; the uppercase calligraphic letters (e.g., $\mathcal{A}$, $\mathcal{B}$, $\mathcal{C}$) as well as accents (e.g., $\bar{L}$, $\bar{\zeta}$) to denote sets; and when convenient, the corresponding unaccented symbols denote elements ranging over those sets (e.g., $\zeta \in \bar{\zeta}$). For a set $\mathcal{S}$ and $m \in \mathbb{N}$, $\mathcal{P}_{\leq m}(\mathcal{S})$ denotes the set of subsets of $\mathcal{S}$ whose cardinality is at most m , i.e., $\mathcal{P}_{\leq m}(\mathcal{S}) \triangleq \{C \subseteq \mathcal{S} \mid |C| \leq m\}$. The metafunction $\pi_k(\cdot)$ for a natural number k , denotes access to the k^{th} component of a tuple. When applied to a set or a bag of tuples, it lifts pointwise and returns the set or bag of the k^{th} components of those tuples, respectively. $\square$

2.1 The Property Graph Data Model

The property graph data model [1, 26] is richer than the edge-labeled graphs typically used in theoretical work on regular path queries [5]: nodes and edges carry *labels* describing their domain role, *properties* stored as key–value pairs, and *directionality* markers that distinguish directed from undirected relationships. This additional structure makes GQL’s type system nontrivial with label expressions, property constraints, join conditions, etc., all inspecting graph structure absent in simpler models and abstractions. We formalize it in full without reducing to a minimal abstraction.

Figure 1 shows an example concrete property graph instance. The graph contains eight nodes carrying one of three node labels: five Person nodes $\{a, c, e, o, z\}$, two Company nodes $\{x, u\}$, and one Project node $\{s\}$. Edges carry one of five labels: KNOWS, WORKS, LEADS, FUNDS, and INVESTS, forming a multigraph with parallel edges and cycles. The graph is mixed and contains directed edges (e.g., the WORKS edge in $o \rightarrow u$) as well as undirected edges (e.g., the KNOWS edge in $e - \text{KNOWS} - z$). Edge labels denote the semantic role of the relationship connecting two nodes, and two nodes may be connected by more than one edge (multigraphs). Many nodes also carry property maps, like the Project node s : $\{\text{name} \mapsto \text{“Atlas”}, \text{projectID} \mapsto 451\}$. These properties illustrate the key feature of property graphs: nodes and edges may carry arbitrary and finite (possibly empty) key–value maps whose values range over primitive domains such as integers, strings, and booleans; they are used for capturing the attributes of the data being modeled.

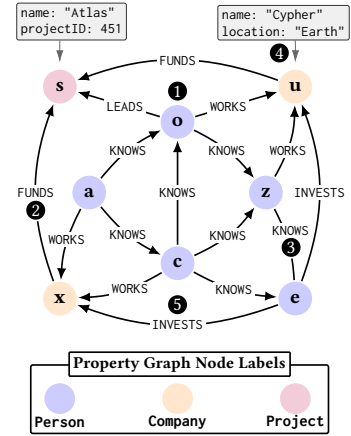


Fig. 1. An example of a property graph instance that contains: nodes (①), directed edges (②), undirected edges (③), node property maps (④), and labeled edges (⑤).

Graph Instance. We assume a multi-sorted universe of values $\mathcal{U}$ given as the disjoint union of sub-universes of the types $\mathcal{T}_0 \triangleq \mathbb{Z} \mid \mathbb{S} \mid \mathbb{B}$; with $\mathbb{Z}$, $\mathbb{S}$, and $\mathbb{B}$ denoting the universes of integers, strings, and booleans, respectively. We also assume an infinite universe of named identifiers $\mathcal{A}$. A *property map* $\Pi: \mathcal{A} \rightarrow \mathcal{U}$ is a finite partial map from elements (*keys*) in $\mathcal{A}$ to those (*values*) in $\mathcal{U}$.

Definition 2.1 (Property Graph Instance). A *property graph* $\mathcal{G}$ is a tuple $(\mathcal{N}, \mathcal{E}, \Delta, \Theta, \Lambda, \Xi)$ where:

- (1) $\mathcal{N}$ and $\mathcal{E}$ are finite sets of *nodes* and *edges*, respectively.
- (2) $\Delta: \mathcal{E} \rightarrow \{\cup, \otimes\}$, gives the *directionality* of edges; whether directed ($\cup$) or undirected ($\otimes$).

- (3) $\mathcal{E} = \mathcal{E}^\cup \sqcup \mathcal{E}^\bowtie$, the set of edges $\mathcal{E}$ can be partitioned into sets $\mathcal{E}^\cup$ and $\mathcal{E}^\bowtie$ based on directionality.
- (4) $\Theta: \mathcal{E}^\cup \sqcup \mathcal{E}^\bowtie \rightarrow (\mathcal{N} \times \mathcal{N}) \sqcup \binom{\mathcal{N}}{2}$, maps edges to their endpoint nodes. For $e \in \mathcal{E}^\cup$, $\Theta(e) = (n_1, n_2) \in \mathcal{N} \times \mathcal{N}$ is an *ordered pair* with n_1 and n_2 referred to as its *source* and *target* nodes. For $e \in \mathcal{E}^\bowtie$, $\Theta(e) = \{n_1, n_2\} \in \binom{\mathcal{N}}{2}$ is an *unordered pair*. Self-loops are permitted, i.e., $n_1 = n_2$.
- (5) $\Lambda: \mathcal{N} \cup \mathcal{E} \rightarrow \mathcal{P}_{\leq m}(\mathcal{A})$, $m \in \mathbb{N}$; maps nodes/edges to finite (maybe empty) sets of *labels* from $\mathcal{A}$.
- (6) $\Xi: \mathcal{N} \cup \mathcal{E} \rightarrow \Pi$, maps nodes/edges to their properties. Π may be empty, i.e., $\mathbf{dom}(\Xi(\cdot)) = \emptyset$.

Graph Schemas. The GQL standard permits property graphs to be associated with *graph schemas* [4], which constrain the admissible labels and properties on nodes and edges (ISO §4.13.2).

Definition 2.2 (Property, Node, Edge, and Graph Schemas). Let $\bar{L} \subset \mathcal{A}$ be a set of labels.

- [Φ]. A *property schema* (ISO §4.13.2.5) $\Phi: \mathcal{A} \rightarrow \mathcal{T}_0$ is a finite partial, mapping keys to scalar types.
- [ζ]. A *node schema* (ISO §4.13.2.3) $\zeta \triangleq (\bar{L}, \Phi)$ is a pair of a set of labels $\bar{L}$ and a property schema Φ .
- [ξ]. An *edge schema* (ISO §4.13.2.4) $\xi \triangleq (\bar{L}, \Phi, \zeta_1, \zeta_2, \delta)$ is a tuple of a set of labels $\bar{L}$, a property schema Φ , its endpoint node schemas ζ_1 and ζ_2 , and edge directionality $\delta \in \{\cup, \bowtie\}$.
 - For $\delta = \cup$, ζ_1 and ζ_2 correspond to the source and target node schemas, respectively.
 - For $\delta = \bowtie$, ζ_1 and ζ_2 are unordered endpoint schemas.
- [Ψ]. A *graph schema* (ISO §4.13.2.2) $\Psi \triangleq (\bar{\zeta}, \bar{\xi})$ is a pair of finite sets of node $\bar{\zeta}$ and edge $\bar{\xi}$ schemas.

Definition 2.3 (Conformance Relations). Let $\mathcal{G} = (\mathcal{N}, \mathcal{E}, \Lambda, \Theta, \Lambda, \Xi)$ be a property graph, $\Psi = (\bar{\zeta}, \bar{\xi})$ a graph schema, and $\Phi: \mathcal{A} \rightarrow \mathcal{T}_0$ a property schema. Suppose $\zeta \in \bar{\zeta}$ and $\xi \in \bar{\xi}$.

- [$\Pi \models \Phi$]. A property map Π *conforms* to Φ when $\mathbf{dom}(\Pi) = \mathbf{dom}(\Phi) \wedge \forall k \in \mathbf{dom}(\Pi). \Pi(k) \in \mathcal{U}_{\Phi(k)}$.
- [$n \models_{\mathcal{G}}^n \zeta$]. A node $n \in \mathcal{N}$ *conforms* to a node schema $\zeta = (\bar{L}, \Phi)$, when $\Lambda(n) = \bar{L}$ and $\Xi(n) \models \Phi$.
- [$e \models_{\mathcal{G}}^e \xi$]. An edge $e \in \mathcal{E}$ *conforms* to an edge schema $\xi = (\bar{L}, \Phi, \zeta_1, \zeta_2, \delta)$ when $\Lambda(e) = \bar{L}$, $\Xi(e) \models \Phi$, and:
 - $e \in \mathcal{E}^\cup$ for $\delta = \cup$ with $n_1 \models_{\mathcal{G}}^n \zeta_1$ and $n_2 \models_{\mathcal{G}}^n \zeta_2$ where $\Theta(e) = (n_1, n_2)$; or
 - $e \in \mathcal{E}^\bowtie$ for $\delta = \bowtie$ with $\exists i, j \in \{1, 2\}. i \neq j. n_i \models_{\mathcal{G}}^n \zeta_i$ and $n_j \models_{\mathcal{G}}^n \zeta_j$ where $\Theta(e) = \{n_1, n_2\}$.
- [$\mathcal{G} \models \Psi$]. The graph $\mathcal{G}$ *conforms* to Ψ when $\forall n \in \mathcal{N}. \exists \zeta \in \bar{\zeta}. n \models_{\mathcal{G}}^n \zeta$ and $\forall e \in \mathcal{E}. \exists \xi \in \bar{\xi}. e \models_{\mathcal{G}}^e \xi$.

Definition 2.4 (Closed and Open Property Graphs). Let $\bar{\Psi}$ be a set of graph schemas and $\mathcal{G}$ a property graph. $\mathcal{G}$ is defined as *closed* or *open* under $\bar{\Psi}$ based on whether $\exists \Psi \in \bar{\Psi}. \mathcal{G} \models \Psi$ or $\nexists \Psi \in \bar{\Psi}. \mathcal{G} \models \Psi$, respectively, i.e., closed graphs conform to atleast one schema, while open graphs conform to none.

The ISO standard refers to the node, edge, and graph schemas from Definition 2.2 as their respective *types*. This alias is evident from the conformance relations (Definition 2.3), where a graph or a graph element, i.e., node or edge, *conforms* to a schema only when it satisfies the schema's constraints on all of its semantic attributes, i.e., labels, properties, directionality, etc., which can be likened to fields. Therefore, when a graph conforms to a schema, i.e., closed graphs, the schema can be leveraged to type GQL queries operating on that graph more precisely, i.e., assign narrower types. We design our type system (§3) to exploit the type information inferable from the schemas.

2.2 GQL

GQL [14, 20] is the first international standard for a property-graph query language, standardized in 2024 as ISO/IEC 39075; and aimed at unifying ideas and concepts from other popular graph query languages such as Cypher [15], PGQL [30], and G-CORE [2], etc., into a single and vendor-neutral specification. Similar to relational query languages like SQL, a GQL query specifies *what* to retrieve from data sources, and optionally transform them before retrieving; while the query

Query	
q	$::= Q \mid q \otimes Q$ composite query (ISO §14.2)
Q	$::= \text{USE } x \text{ MATCH } p \text{ WHERE } \phi \text{ RETURN } \mu$ focused linear query (ISO §14.3) [USE $\rightarrow$ GQ01]
$\otimes$	$::= \text{OTHERWISE} \mid \text{UNION}$ composite ops. (ISO §14.2) [OTHERWISE $\rightarrow$ GQ02, UNION $\rightarrow$ GQ03]
	$\mid \text{EXCEPT DISTINCT} \mid \text{INTERSECT DISTINCT}$ (distinct) [EXCEPT DISTINCT $\rightarrow$ GQ04, INTERSECT DISTINCT $\rightarrow$ GQ06]
	$\mid \text{EXCEPT ALL} \mid \text{INTERSECT ALL}$ (not distinct) [EXCEPT ALL $\rightarrow$ GQ05, INTERSECT ALL $\rightarrow$ GQ07]
μ	$::= _ \mu \mid \mu, _ \mu$ projection (ISO §14.11)
Pattern	
p	$::= P \mid p, P$ path pattern list (ISO §16.4) [, $\rightarrow$ G030]
P	$::= N \mid PEN \mid (P)$ parenthesized path pattern (ISO §16.7) [(P) $\rightarrow$ G038]
	$\mid PEKN \mid (P)K$ quantified patterns (ISO §16.7) [(P)K $\rightarrow$ G035, EK $\rightarrow$ G036]
K	$::= * \mid + \mid ? \mid \{i\} \mid \{i, j\}$ quantifiers (ISO §16.11) [* , + , ? $\rightarrow$ G061; {i} , {i, j} $\rightarrow$ G060]
Pattern Atom	
N	$::= (\text{Dsc})$ node atom (ISO §16.7)
E	$::= \text{---}[\text{Dsc}] \text{---} \mid \text{---}[\text{Dsc}] \text{---} \mid \text{---}[\text{Dsc}] \text{---}$ directed edge atom (ISO §16.7)
	$\mid \text{---}[\text{Dsc}] \text{---} \mid \text{---}[\text{Dsc}] \text{---} \mid \text{---}[\text{Dsc}] \text{---}$ constrained directed or undirected edge atom (ISO §16.7)
	$\mid \text{---}[\text{Dsc}] \text{---}$ any edge atom (ISO §16.7)
Atom Descriptor	
Dsc	$::= \text{VarDcl} \mid \text{LblDcl} \mid \text{PrpDcl}$ atom descriptors
VarDcl	$::= \epsilon \mid x$ pattern variable declaration (ISO §21.1)
LblDcl	$::= \epsilon \mid :l$ label declaration (ISO §16.7)
PrpDcl	$::= \epsilon \mid \{ \Pi \}$ property declaration (ISO §16.7)
l	$::= _l \mid l \& _l \mid l _l \mid !l$ label expressions (ISO §16.8)
Π	$::= x : c \mid \Pi, x : c$ property map (ISO §16.7)
Expression	
ϑ	$::= _ \vartheta \mid \vartheta \oplus _ \vartheta \mid \phi \mid \alpha$ value expressions (ISO §20)
ϕ	$::= _ \phi \mid \text{NOT } \phi \mid \phi \otimes _ \phi$ predicate expressions (ISO §19)
α	$::= _ \alpha(\vartheta) \mid _ \alpha(\text{DISTINCT } \vartheta) \mid _ \alpha(\text{ALL } \vartheta)$ aggregate functions (ISO §20.9)
Helpers	
$_ \vartheta$	$::= c \mid \text{NULL} \mid x.x \mid x$ value atoms (ISO §20.11, 21)
$_ \phi$	$::= \text{TRUE} \mid \text{FALSE} \mid \vartheta \otimes _ \vartheta \mid \vartheta \text{ ISNULL}$ predicate atoms (ISO §19.3, 19.5)
$_ l$	$::= \% \mid x$ label atoms (ISO §16.8) [% $\rightarrow$ G074]
$_ \alpha$	$::= \text{COUNT} \mid \text{SUM} \mid \text{MAX} \mid \text{MIN}$ aggregation atoms
$_ \mu$	$::= x \mid \vartheta \text{ AS } x$ projection atoms
$\oplus$	$::= + \mid - \mid * \mid /$ arithmetic operators
$\otimes$	$::= < \mid <= \mid > \mid >= \mid = \mid <>$ relational operators
$\otimes$	$::= \text{AND} \mid \text{OR}$ binary logical operators

Fig. 2. The formalized GQL calculus with the relevant ISO sections and optional features (GXXX), described in gray. $x \in \mathcal{A}$; $i, j \in \mathbb{N}$; $\mathcal{U}_B = \{\text{TRUE}, \text{FALSE}\}$; and $c \in \mathcal{U}_\tau$, $\tau \in \mathcal{T}_0$. We use the metavariable A to denote N and E .

engine determines *how* to evaluate the query. Broadly, data sources in query languages can be divided into constructed sources and stored sources. Property graphs serve as stored data sources in graph query languages, while *base tables* [19] are an example of stored sources in SQL. Since base tables are highly structured data sources with rigid schemas, a fragment supporting just column projections is often sufficient purely for data retrieval in SQL lineage of languages. Property graphs on the other hand have far more flexible schemas, and so graph query languages include a rich graph pattern matching fragment specifically for data retrieval. The fragment for the optional data transformations before retrieving, largely overlap between relational and graph query languages.

The interplay between the graph pattern matching fragment for data retrieval and the remaining largely relational fragment for data transformations, make GQL’s semantics non-trivial. The semantics is further complicated by GQL allowing querying multiple graphs within a single query; with different ways of combining their results, via composite queries and operators. We formalize a GQL fragment that captures the difficult semantic challenges arising from the interplay.

Formalized GQL Fragment. Figure 2 shows the GQL fragment we formalize. A GQL query q is either a focused linear query Q , i.e., a query operating on a single graph instance, without any composite query operators; or a composite query $q \otimes Q$ that combines the results of two or more sub-queries using composite query operators $\otimes \in \{\text{OTHERWISE}, \text{UNION}, \text{EXCEPT}, \text{INTERSECT}\}$.

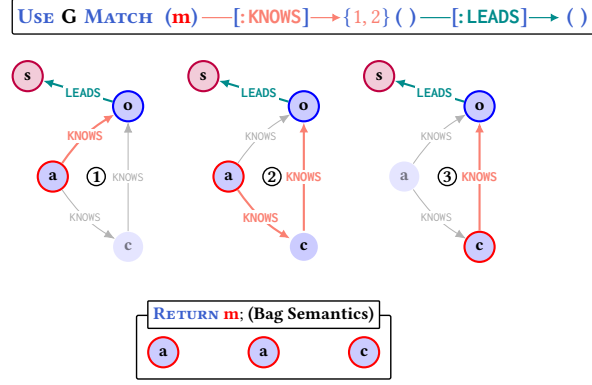


Fig. 3. An example of graph pattern matching with GQL.

A focused linear query Q first declares the graph being queried using the **USE** x clause, where $x \in \mathcal{A}$ refers to the name assigned to the graph instance in the database's catalog. We use **G** specifically to refer to graph names, i.e., **USE G** selects the graph instance named **G** in the catalog as the graph being queried (also called the working graph). Declaration of the working graph is followed by a **MATCH** p clause which is the heart of GQL's graph pattern matching fragment. It specifies a pattern list p . Next, an optional **WHERE** ϕ clause for filtering the matched bindings using the predicate expression ϕ . Finally, a **RETURN** μ clause for returning the query result after projecting the filtered matched bindings via the list of projections μ .

A pattern list p is built from path patterns P composed via conjunction p, P : a comma-separated list requiring *all* patterns to match. A path pattern P is a sequence of node atoms N connected by—only directed ($\longrightarrow$, $\longleftarrow$, $\longleftrightarrow$), right directed or undirected ($\rightsquigarrow$), left directed or undirected ($\leftrightsquigarrow$), only undirected ($\sim$), and unconstrained (---)—edge atoms E . Edges and patterns can be quantified by quantifiers K whose semantics parallels that of regular-expression repetition. The quantifier K includes: Kleene star ($*$), Kleene plus ($+$), optionality ($?$), exact repetition ($\{i\}$), and bounded repetition ($\{i, j\}$), where $i, j \in \mathbb{N}$ and $i \leq j$. Node atoms N may declare binding variables x , specify a label expression l , and impose property constraints via a property map PrpDcl . Same holds for the edge atoms E . Label expressions l are formed from label names ($L \in \mathcal{A}$) and the wildcard ($\%$) under: conjunction ($\&$), disjunction ($|$), and negation ($!$). This allows for fine-grained control over which graph elements a pattern atom matches.

Value expressions ϑ include constants, variables, property accesses $x.x$, arithmetic, and aggregate function applications α . Predicates ϕ are formed from Boolean constants **TRUE/FALSE**, relational operations $\vartheta \odot \vartheta$, the null test $\vartheta \text{ ISNULL}$, and the logical connectives (**AND**, **OR**, **NOT**). Aggregate functions **COUNT**, **SUM**, **MAX**, and **MIN** may be applied with a **DISTINCT** or **ALL** (default) qualifier, which decides if the aggregating operation should be performed under set or bag semantics.

Execution Constructs. A GQL query executes within a *database world* $\Sigma = (\Omega, \Upsilon)$. The *graph catalog* $\Omega: \mathcal{A} \rightarrow \mathcal{G}$ is a fixed collection that resolves graph names ($G \in \mathcal{A}$) to property graph instances (ISO §4.2.5). The *schema mapper* $\Upsilon: \mathcal{A} \rightarrow \Psi$ associates each graph name with a graph schema (if defined). Schema information is available only for closed graphs. The *working graph site* G designates the graph currently in scope and is selected from the catalog via the **USE** clause. All pattern matching in the query after graph selection operates against that graph (ISO §4.7.5).

We formally define GQL's execution constructs when formalizing the semantics (§5). Here, we introduce them informally for contextual reasons. A *record* is a finite set of fields, each pairing a variable name with a runtime value (§5.1)—a scalar, a graph-element identity, or the null value (ISO §4.15.4). A *binding table* is a bag of records having compatible schema (Definition 3.3). It serves as the primary execution construct in query evaluation pipeline, by holding intermediary results such as the matches found by graph pattern matching, and constituting the execution result returned after query evaluation (ISO §4.3.6). Every clause in a query transforms a binding table: **MATCH** produces one, **WHERE** filters it, and **RETURN** projects it into the query's result.

Evaluating **MATCH** p against the working graph enumerates all graph homomorphisms $h: p \rightarrow \mathcal{G}$ that satisfy the structural, label, and property constraints of p . Each such homomorphism yields a record that binds the pattern variables to the matched graph elements, and the collection of all such records forms the binding table produced by the **MATCH** clause. Pattern conjunction combines binding tables via a join on shared variables.

The **WHERE** clause retains only those records for which the predicate ϕ evaluates to true, and the **RETURN** clause maps each surviving record to its projected expressions, producing the output binding table of the query. Composite queries compose queries by combining their binding tables under different composite query operators: **UNION**—union, **INTERSECT**—intersection, **EXCEPT**—difference, and **OTHERWISE** returns the left operand's table when non-empty otherwise the right's.

Example 2.1 (GQL Graph Pattern Matching). Consider the example GQL query shown in Figure 3:

USE G **MATCH** $(m) \text{---} [: \text{KNOWS}] \rightarrow \{1, 2\}() \text{---} [: \text{LEADS}] \rightarrow ()$ **RETURN** m ;

This query has the shape **MATCH** P **RETURN** μ , from the core calculus in Figure 2. The projection item is $\mu = m$ and a single path pattern $P = (m) \text{---} [: \text{KNOWS}] \rightarrow \{1, 2\}() \text{---} [: \text{LEADS}] \rightarrow ()$. The pattern P requires a node m from which there exists a path of one or two KNOWS edges to some intermediate node, followed by a single outgoing LEADS edge to a final node. Evaluation against the property graph instance $\mathcal{G}$ in Figure 1 proceeds by enumerating all graph homomorphisms $h: P \rightarrow \mathcal{G}$ satisfying the path constraint, yielding three records in the binding table:

- ① $m = a$, via the path $a \text{---KNOWS} \rightarrow o \text{---LEADS} \rightarrow s$.
- ② $m = a$, via the longer path $a \text{---KNOWS} \rightarrow c \text{---KNOWS} \rightarrow o \text{---LEADS} \rightarrow s$ from quantifier $\{1, 2\}$.
- ③ $m = c$, via the path $c \text{---KNOWS} \rightarrow o \text{---LEADS} \rightarrow s$.

Since no **WHERE** clause is present, all three records survive filtering. The **RETURN** m clause then projects each record onto the variable m , and under bag semantics the result is the bag $\{a, a, c\}$ containing one entry per homomorphism, including the duplicate binding of m to a . $\square$

Metafunctions for Parsing Patterns. Our formalization uses several metafunctions for parsing GQL query patterns as shown in Figure 4. **var**($\cdot$), **lbl**($\cdot$), and **prp**($\cdot$) return the binding variable, label expression, and property map of a pattern atom (node or edge). We interpret the property map Π returned by **prp**($\cdot$) as a partial, mapping property names to values. **lo**($\cdot$) returns the lower bound of a quantifier. **tail**($\cdot$) returns the binding variable of a pattern's rightmost node atom. **lo**^{#E}($\cdot$) returns the minimum number of edge atoms in a pattern, while **lo**^{#N}($\cdot$) returns the minimum number of node atoms in a pattern. Both account for quantifiers. Finally, **ornt**($\cdot$) maps an edge atom based on its direction, to a set of edge *orientations*: left directed ($\leftarrow$), undirected ($\sim$), and right directed ($\rightarrow$); while **dir**($\cdot$) maps orientations to directionalities: directed ($\hookrightarrow$) or undirected ($\otimes$).

Atom Descriptor	Pattern	Edge Orientations
$\text{var}(\text{Dsc}) \triangleq \text{VarDcl}$ $\text{lbl}(\text{Dsc}) \triangleq \begin{cases} l & \text{if } \text{LblDcl} = :l \\ \epsilon & \text{otherwise} \end{cases}$ $\text{prp}(\text{Dsc}) \triangleq \begin{cases} \Pi & \text{if } \text{PrpDcl} = \{\Pi\} \\ \emptyset & \text{otherwise} \end{cases}$	$\text{tail}(P) \triangleq \begin{cases} \text{var}(N) & \text{if } P=N \mid P'EN \mid P'EKN \\ \text{tail}(P') & \text{if } P=(P') \mid (P')K \end{cases}$ $\text{lo}^{\#E}(P) \triangleq \begin{cases} 0 & \text{if } P=N \\ \text{lo}^{\#E}(P') + 1 & \text{if } P=P'EN \\ \text{lo}^{\#E}(P') + \text{lo}(K) & \text{if } P=P'EKN \\ \text{lo}^{\#E}(P') & \text{if } P=(P') \\ \text{lo}^{\#E}(P') \times \text{lo}(K) & \text{if } P=(P')K \end{cases}$ $\text{lo}^{\#N}(P) \triangleq \begin{cases} 1 & \text{if } P=N \\ \text{lo}^{\#N}(P') + 1 & \text{if } P=P'EN \\ \text{lo}^{\#N}(P') + 1 & \text{if } P=P'EKN \\ \text{lo}^{\#N}(P') & \text{if } P=(P') \\ \text{lo}^{\#N}(P') \times \text{lo}(K) & \text{if } P=(P')K \end{cases}$	$\text{ornt}(E) \triangleq \begin{cases} \{\rightarrow\} & \text{if } E = \text{---} \cdot \rightarrow \\ \{\leftarrow\} & \text{if } E = \leftarrow \cdot \text{---} \\ \{\sim\} & \text{if } E = \text{---} \cdot \text{---} \\ \{\leftarrow, \rightarrow\} & \text{if } E = \leftarrow \cdot \rightarrow \\ \{\sim, \rightarrow\} & \text{if } E = \text{---} \cdot \text{---} \\ \{\leftarrow, \sim\} & \text{if } E = \text{---} \cdot \text{---} \\ \{\leftarrow, \sim, \rightarrow\} & \text{if } E = \text{---} \cdot \text{---} \end{cases}$
Quantifier Lower Bounds		Orientation to Directionality
$\text{lo}(K) \triangleq \begin{cases} 0 & \text{if } K = * \\ 1 & \text{if } K = + \\ 0 & \text{if } K = ? \\ i & \text{if } K = \{i\} \\ i & \text{if } K = \{i, j\} \end{cases}$		$\text{dir}(\partial) \triangleq \begin{cases} \cup & \text{if } \partial \in \{\leftarrow, \rightarrow\} \\ \otimes & \text{if } \partial = \sim \end{cases}$ where $\partial \in \{\leftarrow, \sim, \rightarrow\}$

Fig. 4. Metafunctions for parsing GQL’s graph pattern matching fragment. *Atom Descriptor* metafunctions **var**, **lbl** and **prp**; lift to pattern atoms N and E via their descriptors (Dsc). Only the quantifier lower bounds $\text{lo}(K)$ are given since interpreting upper bounds requires a specific property graph instance (for TRAIL mode).

3 Type System

The graph schema support intrinsic to GQL’s standard allows for more precise typing. We design our type system to exploit the availability of graph schemas to propagate their graph-structural invariants through query patterns into query results. Broadly, our design includes four features of the GQL specification, some of which are absent from simpler formalisms [11, 13]: (1) *graph scoping*—elements of different graphs cannot be conflated with each other; (2) *schema refinement*—closed graphs carry schemas that constrain semantic attributes of their elements allowing for more precise typing; (3) *nullability*—missing properties and failed matches produce null values which obey three-valued logic (3VL); (4) and *heterogeneous unions*—composite queries and property accesses produce bindings whose runtime type varies across records within the same binding table.

$\mathcal{T}_0 ::= \mathbb{Z}$ integer	$\mathcal{T}_1 ::= \mathcal{T}_0$ scalars	$\mathcal{T}_2 ::= \mathcal{T}_1$ value types
$\mathbb{B}$ boolean	$\mathbf{N}(\rightarrow)$ node	Any top type
$\mathbb{S}$ string	$\mathbf{N}(\rightarrow, -)$ node	$\perp$ bottom type
	$\mathbf{E}(\rightarrow)$ edge	$?$ null type
	$\mathbf{E}(\rightarrow, -)$ edge	$\mathcal{T}_2 \cup \mathcal{T}_2 \mid \text{List } \mathcal{T}_2 \mid \mathcal{T}_2^\perp$ type formers

Our type system is organized into three tiers. $\mathcal{T}_0$ gives *scalar types* (property-value primitives). $\mathcal{T}_1$ extends scalars with *graph-scoped element types*—node and edge identity types indexed by a graph site and optionally refined by a schema entry. $\mathcal{T}_2$ gives the full *value type* language: element types $\mathcal{T}_1$, the top type **Any**, the bottom type $\perp$, the null type $?$, and is closed under the types formed from the union, the list, and the empty $(\cdot)^\perp$ formers. Unless otherwise stated, we use τ to range over $\mathcal{T}_2$.

We now motivate our types and type formers, starting with graph-indexed and schema-refined types (§3.1), followed by the union former (§3.2), then the list former (§3.3), and finally, the null type and the empty former (§3.4). We conclude with the subtyping rules for our type system (§3.5).

3.1 Graph-Indexed and Schema-Refined Types

GQL supports composite queries where the individual queries may query different property graphs. However, graph elements are inherently *graph-local*, i.e., nodes and edges of one property graph

cannot be conflated with another. We prevent cross-graph element conflation through *graph-indexed types*: for a graph site G , the types $\mathbf{N}\langle G \rangle$ and $\mathbf{E}\langle G \rangle$ classify node and edge identities scoped to G .

For graph sites referring to closed graphs (Definition 2.4), the type system refines graph-indexed types with schema entries. The schema-refined types $\mathbf{N}\langle G, \zeta \rangle$ and $\mathbf{E}\langle G, \xi \rangle$ classify elements whose labels, properties, etc., conform (Definition 2.3) to their respective schema entries. Refined types carry strictly more static information, enabling more precise type assignments and propagations.

3.2 GQL's Dynamic Union Types: Union Type Former

Example 3.1 (Heterogeneous Unions). Consider the composite GQL query,

$\text{USE } G \text{ MATCH } (v_1:\text{COMPANY}) \text{ RETURN } v_1 \text{ AS } x$	$\text{UNION USE } G \text{ MATCH } (v_2:\text{PROJECT}) \text{ RETURN } v_2 \text{ AS } x$
Returns a binding table with column x typed as $\mathbf{N}\langle G, \zeta_1 \rangle$	Returns a binding table with column x typed as $\mathbf{N}\langle G, \zeta_2 \rangle$

evaluated against the property graph (assuming closed) from Figure 1. Suppose the two individual focused linear queries assign different types ($\mathbf{N}\langle G, \zeta_1 \rangle$ and $\mathbf{N}\langle G, \zeta_2 \rangle$) to the column x in their respective binding tables; the combined binding table must classify x as inhabiting *either* type. $\square$

Example 3.1 illustrates the need for supporting heterogeneous type unions due to GQL's composite queries. Similar situations may arise from property accesses. We model heterogeneous unions by closing our type system under the union former. For any two types τ_1 and τ_2 , the dynamic union $\tau_1 \cup \tau_2$ is a static type that admits values whose type inhabits either branch, i.e., the union former does not introduce a new kind of runtime value, every inhabiting value belongs to either τ_1 or τ_2 .

Notation 3.1 (Schema-Refined Union Types). Consider a graph site G with graph schema $(\bar{\zeta}, \bar{\xi})$, and some *non-empty* set of node $\bar{\zeta}' \subseteq \bar{\zeta}$ and edge $\bar{\xi}' \subseteq \bar{\xi}$ schemas. We denote the node and the edge schema refined types formed from these sets of node and edge schemas, closed under the union type former, using the syntactic sugars: $\mathbf{N}\langle G, \bar{\zeta}' \rangle \triangleq \bigcup_{\zeta \in \bar{\zeta}'} \mathbf{N}\langle G, \zeta \rangle$ and $\mathbf{E}\langle G, \bar{\xi}' \rangle \triangleq \bigcup_{\xi \in \bar{\xi}'} \mathbf{E}\langle G, \xi \rangle$. $\square$

3.3 List Types and Reference Modes

Binding variables declared inside quantified path patterns in GQL can contribute multiple bindings to a single match (ISO §4.11.5). Consider Example 2.1 with the quantified KNOWS segment annotated with edge variable x : $(\mathbf{m}) \text{ --- } [x:\text{KNOWS}] \rightarrow \{1, 2\}() \text{ --- } [: \text{LEADS}] \rightarrow ()$. The variable $\mathbf{m}$ (outside the quantifier) remains *singleton*, i.e., each match binds it to exactly one node. The variable x (inside the quantifier $\{1, 2\}$) is *group-reference*, i.e., contributes multiple bindings per match: a one-hop match yields $[-\text{KNOWS} \rightarrow]$, a two-hop match yields $[-\text{KNOWS} \rightarrow, -\text{KNOWS} \rightarrow]$. The binding exposed outside the quantifier is not a single edge but the finite list of all edges matched by that segment.

We model this statically via **List** τ : if a binding variable has type τ inside the quantifier, crossing the quantifier boundary lifts it to **List** τ . List types thus serve as the static representation of group-reference bindings. In the example above, x is assigned **List** $\mathbf{E}\langle G \rangle$ outside the quantified segment.

Definition 3.1 (Component Type). The *component type* $[\cdot]$ strips the outermost **List** wrapper, i.e., $[\mathbf{List } \tau] = \tau$. The component type of a non-list type is the type itself.

3.4 GQL's Immaterial Types: Null and Empty

Null values are pervasive in GQL, as in SQL [17]; arising from missing properties, optional or failed matches, and undefined intermediate results, etc. The null type $\mathbf{?}$ is inhabited solely by the value **NULL**. We use the syntactic sugar $\tau \mathbf{?}$ to denote the static type $\tau \cup \mathbf{?}$ formed using the union former.

GQL's standard specifies an empty type $\perp$ which no value inhabits, and motivates it as an opportunity to embed richer statically inferrable information such as knowledge that a binding does not produce any runtime values (e.g., failed matches). The empty type is the bottom type.

We retain GQL's empty type $\perp$, and use it as the bottom type of our type system, but in addition, we also introduce the empty type former $\tau^\perp$ to denote a binding that is statically known to produce no runtime value while retaining the underlying type τ as an annotation (GQL's bare empty type $\perp$ erases that information). We motivate this design choice using the Example 3.2 given below.

Example 3.2 (Empty Former). Consider the query with patterns statically known to fail matching.

USE G MATCH ($\mathbf{x} : \text{COMPANY}$), ($\mathbf{x}$) $\text{---} [: \text{KNOWS}] \text{---} ()$ **RETURN** $\mathbf{x}$ *Type Check Pass* ✓

Typing $\mathbf{x}$ as $\mathbf{N}\langle \mathbf{G} \rangle$ preserves the base type $\mathbf{N}\langle \mathbf{G} \rangle$ but loses the emptiness guarantee, while typing it as $\perp$ captures emptiness but erases the base type $\mathbf{N}\langle \mathbf{G} \rangle$. To understand why erasing the base type is problematic, consider a small change to the query where $\mathbf{x}$ binds to an edge in the right pattern:

USE G MATCH ($\mathbf{x} : \text{COMPANY}$), ($\boxed{\mathbf{x}}$) $\text{---} [\boxed{\mathbf{x}} : \text{KNOWS}] \text{---} ()$ **RETURN** $\mathbf{x}$ *Type Check Fail* ✗

This query must be rejected since $\mathbf{x}$ cannot bind both a node and an edge at the same depth of graph pattern matching. But typing $\mathbf{x}$ as $\perp$ erases the base type ($\mathbf{N}\langle \mathbf{G} \rangle$ in the left pattern vs. $\mathbf{E}\langle \mathbf{G} \rangle$ in the right) required for type checking to detect the inconsistency and reject the query. $\square$

The empty type former $(\cdot)^\perp$ resolves the problem in Example 3.2 by encoding emptiness while preserving the underlying type as an annotation. This allows emptiness to propagate without sacrificing the ability to reject ill-formed queries. We restrict $(\cdot)^\perp$ to the element types $\mathbf{N}\langle - \rangle$ and $\mathbf{E}\langle - \rangle$. We use $\mathbf{N}\langle -, \emptyset \rangle$ and $\mathbf{E}\langle -, \emptyset \rangle$ as syntactic sugars to denote $\mathbf{N}\langle \mathbf{G} \rangle^\perp$ and $\mathbf{E}\langle \mathbf{G} \rangle^\perp$, respectively.

3.5 Subtyping

The subtyping relation $\tau_1 <: \tau_2$ is the least reflexive and transitive relation closed under:

S-REFLEXIVE $\frac{}{\tau <: \tau}$	S-TRANSITIVE $\frac{\tau_1 <: \tau_2 \quad \tau_2 <: \tau_3}{\tau_1 <: \tau_3}$	S-ANY $\frac{}{\tau <: \text{Any}}$	S-EMPTY $\frac{}{\perp <: \tau}$	S-LIST $\frac{}{\text{List } \tau_1 <: \text{List } \tau_2}$
S-UNION-LEFT $\frac{}{\tau_1 <: \tau_1 \cup \tau_2}$	S-UNION-RIGHT $\frac{}{\tau_2 <: \tau_1 \cup \tau_2}$	S-UNION $\frac{\tau_1 <: \tau \quad \tau_2 <: \tau}{\tau_1 \cup \tau_2 <: \tau}$	S-UNION-CONGRUENCE $\frac{\tau_1 <: \tau_1' \quad \tau_2 <: \tau_2'}{\tau_1 \cup \tau_2 <: \tau_1' \cup \tau_2'}$	
S-REF-NODE $\frac{}{\mathbf{N}\langle \mathbf{G}, - \rangle <: \mathbf{N}\langle \mathbf{G} \rangle}$	S-REF-NODE-EMPTY $\frac{}{\mathbf{N}\langle \mathbf{G} \rangle^\perp <: \mathbf{N}\langle \mathbf{G}, - \rangle}$	S-REF-EDGE $\frac{}{\mathbf{E}\langle \mathbf{G}, - \rangle <: \mathbf{E}\langle \mathbf{G} \rangle}$	S-REF-EDGE-EMPTY $\frac{}{\mathbf{E}\langle \mathbf{G} \rangle^\perp <: \mathbf{E}\langle \mathbf{G}, - \rangle}$	

S-REFLEXIVE/S-TRANSITIVE makes the subtyping relation a preorder. S-ANY/S-EMPTY bound the type system with top **Any** and bottom $\perp$ types. S-LIST lifts types to their list types. The union rules—S-UNION-LEFT, S-UNION-RIGHT, S-UNION, and S-UNION-CONGRUENCE—make $\tau_1 \cup \tau_2$ operate as least-upper-bound. The schema rules—S-REF-NODE and S-REF-NODE-EMPTY—connect graph-indexed and schema-refined node element types while bounding this sublattice with the annotated empty type $\mathbf{N}\langle \mathbf{G} \rangle^\perp$ at the bottom. S-REF-EDGE and S-REF-EDGE-EMPTY do the same for edge types.

Definition 3.2 (Type Intersection). Let τ_1 and τ_2 be any two types. We define their intersection, written $\tau_1 \cap \tau_2$, as the least reflexive and transitive relation closed under the following judgements:

INTERSECT-LEFT	INTERSECT-RIGHT	INTERSECT
$\frac{\tau_1 <: \tau_2}{\tau_1 \cap \tau_2 : \tau_1}$	$\frac{\tau_2 <: \tau_1}{\tau_1 \cap \tau_2 : \tau_2}$	$\frac{\tau <: \tau_1 \quad \tau <: \tau_2 \quad \tau_1 \star : \tau_2 \quad \tau_2 \star : \tau_1}{\tau_1 \cap \tau_2 : \tau}$

Lemma 3.1 (Absorption). (1) $\tau \cup \mathbf{Any} = \mathbf{Any}$, (2) $\tau \cup \perp = \tau$, (3) $\tau \cap \mathbf{Any} = \tau$, and (4) $\tau \cap \perp = \perp$.

Lemma 3.2 (Schema-Refined Type Bounds). Consider a graph site G with graph schema $(\bar{\zeta}, \bar{\xi})$. Then $\mathbf{N}\langle G \rangle^\perp <: \mathbf{N}\langle G, \zeta \rangle <: \mathbf{N}\langle G \rangle$ and $\mathbf{E}\langle G \rangle^\perp <: \mathbf{E}\langle G, \xi \rangle <: \mathbf{E}\langle G \rangle$ for $\zeta \in \bar{\zeta}$ and $\xi \in \bar{\xi}$, respectively.

Definition 3.3 (Record and Binding Table Schemas). The schema $\Gamma: \mathcal{A} \rightarrow \tau$ of a record specifies the type for each binding variable in its domain. The schema of a binding table is the union (Definition 3.4) of the schemas of all the records in its support.

Definition 3.4 (Record Schema Unions and Joins). Let Γ_1 and Γ_2 be two record schemas.

$[\Gamma_1 \sim_{\cup} \Gamma_2]$. They are *union compatible* when $\mathbf{dom}(\Gamma_1) = \mathbf{dom}(\Gamma_2)$.

$[\Gamma_1 \sim_{\bowtie} \Gamma_2]$. They are *join compatible* when $\forall x \in \mathbf{dom}(\Gamma_1) \cap \mathbf{dom}(\Gamma_2)$:

- $\Gamma_1(x) <: \mathbf{N}\langle - \rangle$ (or $\mathbf{E}\langle - \rangle$) and $\Gamma_2(x) <: \mathbf{N}\langle - \rangle$ (or $\mathbf{E}\langle - \rangle$); and
- $\Gamma_1(x) \cap \Gamma_2(x) \neq \perp$.

$[\Gamma_1 \cup \Gamma_2]$. If $\Gamma_1 \sim_{\cup} \Gamma_2$, their *union* is: $\Gamma_1 \cup \Gamma_2(x) \triangleq \Gamma_1(x) \cup \Gamma_2(x), x \in \mathbf{dom}(\Gamma_1)$.

$[\Gamma_1 \bowtie \Gamma_2]$. If $\Gamma_1 \sim_{\bowtie} \Gamma_2$, their *join* is:

$$\Gamma_1 \bowtie \Gamma_2(x) \triangleq \begin{cases} \Gamma_1(x) & x \in \mathbf{dom}(\Gamma_1) \setminus \mathbf{dom}(\Gamma_2) \\ \Gamma_2(x) & x \in \mathbf{dom}(\Gamma_2) \setminus \mathbf{dom}(\Gamma_1) \\ \Gamma_1(x) \cap \Gamma_2(x) & x \in \mathbf{dom}(\Gamma_1) \cap \mathbf{dom}(\Gamma_2) \end{cases}$$

4 Well-formedness of GQL Queries

We now use our type system (§3) to define well-formedness rules for GQL queries and assign binding table schemas to their results. GQL's native support for graph schemas (ISO §4.13) allows fine-grained static typing. Schema information flows from typing rules for pattern atoms through patterns and queries, providing static guarantees that are unavailable in schema-free formalisms.

We organize GQL query well-formedness into three layers that mirror the syntactic hierarchy of our calculus in Figure 2: value expressions, patterns (§4.2), and queries (§4.3). This organization simplifies the type-soundness argument (§6) by allowing it to proceed compositionally, lifting guarantees from expressions to patterns and ultimately to queries. We only consider queries where all pattern atoms are named (non-anonymous), i.e., they declare their binding variables: $\mathbf{var}(\mathbf{A}) \neq \epsilon$.

4.1 GQL Value Expressions

Due to space constraints, we only introduce the typing judgement for GQL value expressions here for clarity when referencing them in later sections. We type value expressions under the judgement: $\Sigma; G; \Gamma \vdash_{\diamond}^{\square} \vartheta : \tau \bowtie \mathcal{X}$, which assigns type τ to the value expression ϑ under the context containing: the database world (Σ), a working graph site G , a record schema (Γ), the binding variables used in the expression ($\mathcal{X}$), the permitted number of aggregate functions ($\square \in \{0, 1\}$), and the reference-mode ($\diamond \in \{\mathbb{1}, \star\}$) that captures if the expression is inside or outside a quantifier context.

4.2 GQL Patterns

Typing patterns is one of the most technically challenging aspects of defining well-formedness of GQL queries because they introduce all bindings to graph elements, so their typing must statically

predict the schema that the operational semantics produces at runtime. Moreover, since our GQL formalism includes graphs with schemas, typing becomes *strictly more precise* than in schema-free approaches. For instance, a binding variable can be statically assigned $\tau^\perp$ when the schema's label constraints together with the pattern rule out any valid homomorphism. These refinements compose in non-trivial ways because: (a) *surrounding context*—a pattern list is composed of several path patterns that may have overlaps in their declared binding variables, and (b) *internal context*—a pattern is composed of several pattern atoms that may similarly have overlaps in their declared binding variables. We reconcile the two by structuring our typing rules across different levels that mirror the grammar of Figure 2: (1) *atom typing* types individual node and edge atoms in isolation from all context; (2) *path pattern typing* composes atoms only under internal context, staying isolated from surrounding context; and (3) *pattern list typing* composes path patterns via conjunction under surrounding context. Levels (2) and (3) refine overlapping variables left-to-right in one forward pass without fixed-point iteration, which may assign wider types than necessary. We formalize pattern well-formedness using three main judgement forms.

$$\Sigma; G \vdash_{\text{Atom}} A : \Gamma \quad \Sigma; G \vdash_{\text{Pat}}^\diamond P : \Gamma \quad \Sigma; G \vdash_{\text{PatLst}} p : \Gamma$$

The *atom judgement* types a pattern atom A , i.e., a node N or an edge E atom, in isolation to produce a singleton binding table schema Γ . The *pattern judgement* composes atoms into path patterns. Its reference mode $\diamond \in \{\mathbb{1}, \star\}$ records whether the pattern sits outside ($\mathbb{1}$) or inside ($\star$) a quantifier context. The *pattern list judgement* composes path patterns using conjunction (p, P).

Atom Typing (Level 1). Atom typing resolves the label expression l and property map Π of a pattern atom A against the graph schema (ζ, ξ) , producing a schema-refined type. Two auxiliary judgements perform the filtering and operate uniformly on node and edge schemas, so we write ζ for a schema entry of either kind. The judgement $\bar{\zeta} \vdash_{\text{Lbl}} \text{lbl}(A) \rightsquigarrow \bar{\zeta}'$ filters element schemas by structural induction on the label expression, covering: labels L , the wildcard ($\%$), conjunction ($\&$), disjunction ($\mid$), and negation ($!$). The judgement $\bar{\zeta} \vdash_{\text{Prp}} \text{prp}(A) \rightsquigarrow \bar{\zeta}'$ similarly filters element schemas, but by property maps. Intersecting their results yields the compatible element schemas.

SCH-LBL-EMPTY $\frac{}{\bar{\zeta} \vdash_{\text{Lbl}} \varepsilon \rightsquigarrow \bar{\zeta}}$	SCH-LBL-ATOM $\frac{\bar{\zeta}_L := \{\zeta \in \bar{\zeta} \mid L \in \pi_1(\zeta)\}}{\bar{\zeta} \vdash_{\text{Lbl}} L \rightsquigarrow \bar{\zeta}_L}$	SCH-LBL-NEGATION $\frac{\bar{\zeta} \vdash_{\text{Lbl}} l \rightsquigarrow \bar{\zeta}_l}{\bar{\zeta} \vdash_{\text{Lbl}} !l \rightsquigarrow \bar{\zeta} \setminus \bar{\zeta}_l}$	SCH-LBL-CONJUNCTION $\frac{\bar{\zeta} \vdash_{\text{Lbl}} l_1 \rightsquigarrow \bar{\zeta}_1 \quad \bar{\zeta} \vdash_{\text{Lbl}} l_2 \rightsquigarrow \bar{\zeta}_2}{\bar{\zeta} \vdash_{\text{Lbl}} l_1 \& l_2 \rightsquigarrow \bar{\zeta}_1 \cap \bar{\zeta}_2}$
SCH-LBL-DISJUNCTION $\frac{\bar{\zeta} \vdash_{\text{Lbl}} l_1 \rightsquigarrow \bar{\zeta}_1 \quad \bar{\zeta} \vdash_{\text{Lbl}} l_2 \rightsquigarrow \bar{\zeta}_2}{\bar{\zeta} \vdash_{\text{Lbl}} l_1 \mid l_2 \rightsquigarrow \bar{\zeta}_1 \cup \bar{\zeta}_2}$	SCH-LBL-WILDCARD $\frac{\bar{\zeta}_\% := \{\zeta \in \bar{\zeta} \mid \pi_1(\zeta) \neq \emptyset\}}{\bar{\zeta} \vdash_{\text{Lbl}} \% \rightsquigarrow \bar{\zeta}_\%}$	SCH-PRP-ATOM $\frac{\bar{\zeta}_\Phi := \{\zeta \in \bar{\zeta} \mid \Phi \subseteq \pi_2(\zeta)\}}{\bar{\zeta} \vdash_{\text{Prp}} \Phi \rightsquigarrow \bar{\zeta}_\Phi}$	

SCH-LBL-EMPTY returns the schema set as is since there is no label expression. SCH-LBL-ATOM retains schemas whose label set contains L , while SCH-LBL-WILDCARD retains schemas with at least one label. SCH-LBL-NEGATION, SCH-LBL-CONJUNCTION, and SCH-LBL-DISJUNCTION filter schemas based on logical operations corresponding to their names. SCH-PRP-ATOM filters by property maps.

PRP-EMPTY $\frac{}{\vdash_{\text{Prp}} \emptyset : \emptyset}$	PRP-ATOM $\frac{k \in \mathcal{A} \quad \tau \in \mathcal{T}_0 \quad c \in \mathcal{U}_\tau}{\vdash_{\text{Prp}} [k \mapsto c] : [k \mapsto \tau]}$	PRP-INSERT $\frac{k \notin \text{dom}(\Pi) \quad \vdash_{\text{Prp}} \Pi : \Phi_\Pi \quad \vdash_{\text{Prp}} [k \mapsto c] : \Phi_k}{\vdash_{\text{Prp}} \Pi \sqcup [k \mapsto c] : \Phi_\Pi \sqcup \Phi_k}$
---	--	--

The judgement $\vdash_{\text{Prp}} \Pi : \Phi$ types a property map Π with a property schema Φ . PRP-EMPTY types the empty map, PRP-ATOM types a singleton map by inferring the constant's scalar type, and PRP-INSERT combines property schemas via disjoint union provided their domains are also disjoint.

$$\begin{array}{c}
\text{SCH-ATOM} \\
\frac{\text{var}(A) \in \mathcal{A} \quad \vdash_{\text{Prp}} \text{prp}(A) : \Phi \quad \bar{\zeta} \vdash_{\text{Lbl}} \text{lbl}(A) \rightsquigarrow \bar{\zeta}_l \quad \bar{\zeta} \vdash_{\text{Prp}} \Phi \rightsquigarrow \bar{\zeta}_\Phi}{\bar{\zeta} \vdash_{\text{Atom}} A \rightsquigarrow \bar{\zeta}_l \cap \bar{\zeta}_\Phi} \\
\\
\text{ATOM-NODE-OPEN} \qquad \text{ATOM-NODE-CLOSED} \\
\frac{G \notin \text{dom}(\Upsilon) \quad \text{var}(N) \in \mathcal{A}}{\Sigma; G \vdash_{\text{Atom}} N : [\text{var}(N) \mapsto \mathbf{N}\langle G \rangle]} \quad \frac{G \in \text{dom}(\Upsilon) \quad (\bar{\zeta}, -) = \Upsilon(G) \quad \bar{\zeta} \vdash_{\text{Atom}} N \rightsquigarrow \bar{\zeta}_N}{\Sigma; G \vdash_{\text{Atom}} N : [\text{var}(N) \mapsto \mathbf{N}\langle G, \bar{\zeta}_N \rangle]} \\
\\
\text{ATOM-EDGE-OPEN} \qquad \text{ATOM-EDGE-CLOSED} \\
\frac{G \notin \text{dom}(\Upsilon) \quad \text{var}(E) \in \mathcal{A}}{\Sigma; G \vdash_{\text{Atom}} E : [\text{var}(E) \mapsto \mathbf{E}\langle G \rangle]} \quad \frac{G \in \text{dom}(\Upsilon) \quad (-, \bar{\xi}) = \Upsilon(G) \quad \bar{\zeta} \vdash_{\text{Atom}} E \rightsquigarrow \bar{\xi}_E}{\Sigma; G \vdash_{\text{Atom}} E : [\text{var}(E) \mapsto \mathbf{E}\langle G, \bar{\xi}_E \rangle]}
\end{array}$$

SCH-ATOM resolves A 's schema by intersecting its label-expression and property map filter results. The atom-typing rules handle open and closed graphs separately. Open graphs (ATOM-NODE-OPEN) have no schema, so the atom's binding variable $\text{var}(N)$ is assigned just the graph-indexed type $\mathbf{N}\langle G \rangle$. For closed graphs (ATOM-NODE-CLOSED), the variable is assigned the schema-refined type $\mathbf{N}\langle G, \bar{\zeta}_N \rangle$. If the atom's label-expression and property map combination does not satisfy the constraints of any node schema in G 's graph schema, i.e., $\bar{\zeta}_N = \emptyset$; the variable is assigned the annotated empty type $\mathbf{N}\langle G \rangle^\perp$ since $\mathbf{N}\langle G, \emptyset \rangle \triangleq \mathbf{N}\langle G \rangle^\perp$ (§ 3.4). Symmetric rules cover edge atoms.

Path Pattern Typing (Level 2). Path pattern typing composes atoms into complete path patterns. When composing a node-edge-node step $N_1 E_2 N_2$, the types assigned to the atoms— N_1 , E_2 , and N_2 —in isolation may be overly permissive. This is because *atom typing* (Level 1) types atoms with refined element schemas, with refinement based solely on their label-expressions and property maps. While this “may be” sufficient for typing the node atoms N_1 and N_2 , it is not the case for the edge atom E_2 , as edge schemas impose additional constraints beyond label sets and property maps: endpoint node schemas and edge directionality (Definition 2.2). Moreover, refining E_2 's type based on the additional constraints usually requires refining the types of its endpoints N_1 and N_2 as well.

We formalize path pattern typing such that type refinement for the composing node-edge-node step is done jointly, allowing typing to prune infeasible triples and thus assign more precise types. For the path pattern PEN , the judgement $\Sigma; G; \Gamma_P; \Gamma_E; \Gamma_N \vdash_{\text{Rfn}} PEN \rightsquigarrow \Gamma'_P; \Gamma'_E; \Gamma'_N$ jointly refines the types of the binding variables corresponding to the rightmost node atom (i.e., $\text{tail}(\cdot)$) of the preceding pattern P , the edge atom E , and the following node atom N ; from their individual table schemas (typed in isolation)— Γ_P , Γ_E , and Γ_N —to produce the type refined schemas Γ'_P , Γ'_E , and Γ'_N .

Definition 4.1 (Type Preserving Update for Schema-Refined Types). Consider a graph site G with graph schema $(\bar{\zeta}, \bar{\xi})$. Let τ be a schema-refined type $\mathbf{N}\langle G, - \rangle$ (or $\mathbf{E}\langle G, - \rangle$) or its list-lifted version; and let $\bar{\zeta}, \bar{\zeta}'$ range over the subsets of $\bar{\zeta}$ (or $\bar{\xi}$). We then define the type preserving update $\tau|_{\bar{\zeta}}$ as:

$$\tau|_{\bar{\zeta}} \triangleq \begin{cases} \text{List}([\tau][\bar{\zeta}/\bar{\zeta}']) & \text{if } \tau = \text{List } \mathbf{N}\langle G, \bar{\zeta}' \rangle \text{ or } \text{List } \mathbf{E}\langle G, \bar{\zeta}' \rangle \\ \tau[\bar{\zeta}/\bar{\zeta}'] & \text{if } \tau = \mathbf{N}\langle G, \bar{\zeta}' \rangle \text{ or } \mathbf{E}\langle G, \bar{\zeta}' \rangle \end{cases}$$

The operation $\tau[\bar{\zeta}/\bar{\zeta}']$ denotes replacing the schema set $\bar{\zeta}'$ of the schema-refined type τ with the set $\bar{\zeta}$. It yields the appropriate annotated empty type when $\bar{\zeta} = \emptyset$ (e.g., $\mathbf{N}\langle G, \bar{\zeta}' \rangle[\emptyset/\bar{\zeta}'] = \mathbf{N}\langle G \rangle^\perp$).

$$\begin{array}{c}
\text{REFINE-OPEN} \\
\frac{G \notin \mathbf{dom}(\Upsilon) \quad \sim_{\bowtie} (\Gamma_P, \Gamma_E, \Gamma_N)}{\Sigma; G; \Gamma_P; \Gamma_E; \Gamma_N \vdash_{\text{Rfn}} \text{PEN} \rightsquigarrow \Gamma_P, \Gamma_E, \Gamma_N} \quad \theta_E(\zeta_1, \xi_2, \zeta_3) \triangleq \bigvee_{\partial \in \text{ornt}(E)} \begin{cases} (\zeta_1, \zeta_3, \cup) = (\pi_3(\xi_2), \pi_4(\xi_2), \pi_5(\xi_2)) & \text{if } \partial = \rightarrow \\ (\zeta_1, \zeta_3, \cup) = (\pi_4(\xi_2), \pi_3(\xi_2), \pi_5(\xi_2)) & \text{if } \partial = \leftarrow \\ (\zeta_1, \zeta_3, \bowtie) = (\pi_3(\xi_2), \pi_4(\xi_2), \pi_5(\xi_2)) & \text{else} \\ \text{or} \\ (\zeta_1, \zeta_3, \bowtie) = (\pi_4(\xi_2), \pi_3(\xi_2), \pi_5(\xi_2)) \end{cases} \\
\\
\text{REFINE-CLOSED} \\
\frac{G \in \mathbf{dom}(\Upsilon) \quad \sim_{\bowtie} (\Gamma_P, \Gamma_E, \Gamma_N) \quad (\tau_1, \tau_2, \tau_3) := (\Gamma_P(\mathbf{tail}(P)), \Gamma_E(\mathbf{var}(E)), \Gamma_N(\mathbf{var}(N)))}{[\tau_1] \simeq_{\tau} \mathbf{N}\langle G, \bar{\zeta}_1 \rangle \quad [\tau_2] \simeq_{\tau} \mathbf{E}\langle G, \bar{\xi}_2 \rangle \quad [\tau_3] \simeq_{\tau} \mathbf{N}\langle G, \bar{\zeta}_3 \rangle \quad \mathcal{S} := \{(\zeta_1, \xi_2, \zeta_3) \in (\bar{\zeta}_1 \times \bar{\xi}_2 \times \bar{\zeta}_3) \mid \theta_E(\zeta_1, \xi_2, \zeta_3)\}}{\Sigma; G; \Gamma_P; \Gamma_E; \Gamma_N \vdash_{\text{Rfn}} \text{PEN} \rightsquigarrow \Gamma_P[\mathbf{tail}(P) \mapsto \tau_1 \upharpoonright_{\pi_1(\mathcal{S})}], \Gamma_E[\mathbf{var}(E) \mapsto \tau_2 \upharpoonright_{\pi_2(\mathcal{S})}], \Gamma_N[\mathbf{var}(N) \mapsto \tau_3 \upharpoonright_{\pi_3(\mathcal{S})}]}
\end{array}$$

REFINE-OPEN handles type refinement in open graphs and simply checks pairwise join compatibility (Definition 3.4, via shorthand: $\sim_{\bowtie}(\dots)$) of the individual schemas corresponding to the preceding pattern P , edge atom E , and following node atom N ; before returning them unchanged. REFINE-CLOSED handles closed graphs. Let τ_1 , τ_2 , and τ_3 correspond to the types assigned in isolation to the binding variables: $\mathbf{tail}(P)$, $\mathbf{var}(E)$, and $\mathbf{var}(N)$, respectively. We first check if the types are the appropriate schema-refined types up to nullability, using the shorthand: $\tau' \simeq_{\tau} \tau \triangleq \tau <: \tau' <: \tau?$. Next, we form the Cartesian product of their schema sets, retaining only the triples $(\zeta_1, \xi_2, \zeta_3)$ that satisfy the *additional constraints* imposed by edge schemas (ξ_2) via the endpoint condition $\theta_E(\zeta_1, \xi_2, \zeta_3)$. The endpoint condition holds when the node-edge-node schema triple is compatible under some orientation in $\text{ornt}(E)$ (Figure 4). Each orientation ($\partial \in \{\rightarrow, \leftarrow, \sim\}$) compares the endpoint node schemas ζ_1 and ζ_3 against that of the edge schema ξ_2 as a tuple, with ordering determined by the orientation; the edge directionality $\pi_5(\xi_2)$ must match $\mathbf{dir}(\partial)$. Orientation “ $\rightarrow$ ” takes the endpoint node schemas in order, “ $\leftarrow$ ” swaps them, but both require the edge to be directed; while “ $\sim$ ” compares them as an unordered pair for undirected edges. Finally, the filtered schema triples are used to update the variables’ types via the type-preserving update from Definition 4.1.

Definition 4.2 (Quantifier Lift). Given a record schema Γ and a quantifier K , the quantifier lift operation $\Gamma \upharpoonright_K$ is defined as: $\forall x \in \mathbf{dom}(\Gamma)$:

$$\Gamma \upharpoonright_K(x) \triangleq \begin{cases} \Gamma(x)? & \text{if } K = ? \\ \mathbf{List} \Gamma(x) & \text{otherwise} \end{cases}$$

Path patterns are described in ISO §16.7, and SR:8 (Page 229) states that patterns may only be quantified if they have a minimum path length of one. We enforce this during path pattern typing via the premise $\mathbf{lo}^{\#E}(P) > 0$. SR:16 (Page 231) imposes another restriction of path patterns having at least one node atom regardless of quantification. We account for this while typing pattern lists.

PAT-NODE

$$\frac{\Sigma; G \vdash_{\text{Atom}} N : \Gamma}{\Sigma; G \vdash_{\text{Pat}} N : \Gamma} \quad \text{PAT-STEP} \quad \frac{\Sigma; G \vdash_{\text{Pat}}^{\diamond} P : \Gamma_P \quad \Sigma; G \vdash_{\text{Pat}} N : \Gamma_N \quad \Sigma; G; \Gamma_P; \Gamma_E; \Gamma_N \vdash_{\text{Rfn}} \text{PEN} \rightsquigarrow \Gamma'_P, \Gamma'_E, \Gamma'_N}{\Sigma; G; \Gamma_E \vdash_{\text{PatStep}}^{\diamond} \text{PEN} : \Gamma'_P \bowtie \Gamma'_E \bowtie \Gamma'_N}$$

PAT-EDGE

$$\frac{\Sigma; G \vdash_{\text{Atom}} E : \Gamma_E \quad \Sigma; G; \Gamma_E \vdash_{\text{PatStep}}^{\diamond} \text{PEN} : \Gamma}{\Sigma; G \vdash_{\text{Pat}}^{\diamond} \text{PEN} : \Gamma}$$

PAT-QUANT-EDGE

$$\frac{\Sigma; G \vdash_{\text{Atom}} E : \Gamma_E \quad \Sigma; G; (\Gamma_E \upharpoonright_K) \vdash_{\text{PatStep}}^{\star} \text{PEN} : \Gamma}{\Sigma; G \vdash_{\text{Pat}}^{\dagger} \text{PEN} : \Gamma}$$

PAT-PAREN-PATH

$$\frac{\Sigma; G \vdash_{\text{Pat}}^{\diamond} P : \Gamma}{\Sigma; G \vdash_{\text{Pat}}^{\diamond} (P) : \Gamma}$$

PAT-QUANT-PATH

$$\frac{\mathbf{lo}^{\#E}(P) > 0 \quad \Sigma; G \vdash_{\text{Pat}}^{\star} (P) : \Gamma}{\Sigma; G \vdash_{\text{Pat}}^{\dagger} (P) K : \Gamma \upharpoonright_K}$$

PAT-NODE lifts a node atom as a path pattern. PAT-STEP is an auxiliary judgement that type refines the binding variables of a path pattern using the refinement rules described earlier. PAT-EDGE

introduces an edge atom, delegating to PAT-STEP. PAT-QUANT-EDGE handles quantified edges by first lifting their singleton table schemas through $(\cdot) \uparrow_K$ from Definition 4.2 before delegating to PAT-STEP (similar to PAT-EDGE). PAT-PAREN-PATH types parenthesized path patterns by simply typing the enclosed pattern. PAT-QUANT-PATH handles quantified path patterns by checking if they have at least one edge atom, and then typing them, before lifting their table schemas through $(\cdot) \uparrow_K$.

Pattern List Typing (Level 3). Pattern list typing composes path patterns via conjunction (p, P) . We account for the minimum node count restriction imposed by SR:16 (Page 231) here, via $\mathbf{lo}^{\#N}(P) > 0$.

$$\frac{\text{PATLIST-LIFT} \quad \mathbf{lo}^{\#N}(P) > 0 \quad \Sigma; G \vdash_{\text{Pat}}^I P : \Gamma}{\Sigma; G \vdash_{\text{PatLst}} P : \Gamma} \quad \frac{\text{PATLIST-CONJUNCTION} \quad \Sigma; G \vdash_{\text{PatLst}} p : \Gamma_p \quad \Sigma; G \vdash_{\text{PatLst}} P : \Gamma_P \quad \Gamma_p \sim \bowtie \Gamma_P}{\Sigma; G \vdash_{\text{PatLst}} p, P : \Gamma_p \bowtie \Gamma_P}$$

PATLIST-LIFT lifts a path pattern into a pattern list if it has at least one node atom. PATLIST-CONJUNCTION types a conjunction. It types both sub-components—the preceding path pattern list p and the following path pattern P —independently, and verifies the join compatibility of their table schemas Γ_p and Γ_P before joining them to produce a single table schema for the conjunction. Since path pattern lists are typed under surrounding context in one forward left-to-right pass, only those variables declared in p that overlap with the ones in P are refined during the schema join $\Gamma_p \bowtie \Gamma_P$.

4.3 GQL Queries

GQL queries are either focused linear queries (Q) that match patterns against a single working graph and optionally filter the matched bindings before projecting them via projection expressions; or composite queries (q) that compose several focused linear queries via composite operators ($\otimes$).

Query typing is relatively simple since much of the complexity has been deliberately offloaded to typing: (1) value expressions (§4.1)—used for typing the predicate and projection expressions in **WHERE** and **RETURN** clauses, respectively; and (2) patterns (§4.2)—used for typing the patterns in **MATCH** clauses. We type GQL queries under three judgement forms,

$$\begin{array}{ccc} \Sigma; G; \Gamma \vdash \mu : \Gamma_\mu & \Sigma \vdash Q : \Gamma & \Sigma \vdash_{\otimes} q : \Gamma \\ \text{Projection} & \text{Query} & \text{Composite Query} \end{array}$$

The *projection judgement* types projection expressions by projecting an upstream table schema in the context of a working graph. The projected table schema is the schema of a focused linear query’s result binding table. The *query judgement* assigns focused linear queries with the schemas of their result binding tables. The *composite query judgement* types composite queries similarly, but under the additional context of the composite operators used for focused linear query composition.

Projection Typing. Projection expressions project upstream table schemas (Γ) using value expressions (§9). Projection expressions are covered in ISO §14.11, and its SR:8 (Page 186) states that they must be aliased via **As** unless they are just references to binding variables ($x \in \text{dom}(\Gamma)$). We choose the fragment for projection expressions and structure its typing rules to match the standard.

$$\begin{array}{ccc} \text{PRJ-ATOM-VALExp} & \text{PRJ-ATOM-VARRef} & \text{PRJ-COMPOSE} \\ \frac{x \in \mathcal{A} \quad \Sigma; G; \Gamma \vdash_{\star}^I \vartheta : \tau \triangleright \mathcal{X}}{\Sigma; G; \Gamma \vdash \vartheta \text{ As } x : [x \mapsto \tau]} & \frac{\Sigma; G; \Gamma \vdash x \text{ As } x : \Gamma_x}{\Sigma; G; \Gamma \vdash x : \Gamma_x} & \frac{\begin{array}{l} \Sigma; G; \Gamma \vdash \mu : \Gamma_\mu \quad \Sigma; G; \Gamma \vdash _ \mu : \Gamma_\mu \\ \text{dom}(\Gamma_\mu) \cap \text{dom}(\Gamma_\mu) = \emptyset \end{array}}{\Sigma; G; \Gamma \vdash \mu, _ \mu : \Gamma_\mu \bowtie \Gamma_\mu} \end{array}$$

PRJ-ATOM-VALExp covers value expressions generally, and so compulsorily includes an alias. It types value expressions in the context of an upstream table schema, and produces a singleton table schema with the alias mapped to that type. Value expressions that are just binding variable

references are handled by PRJ-ATOM-VARREF, which first rewrites them vacuously into aliased projection expressions, before delegating them to PRJ-ATOM-VALEXP. PRJ-COMPOSE covers projection composition by typing their atoms left-to-right, joining their schemas only when they are disjoint.

Linear Query Typing. Linear queries first select a working graph via **USE**, then introduce binding tables via **MATCH** and optionally filter them via **WHERE**, before projecting them using **RETURN**.

$$\begin{array}{c}
 \text{QRY-MATCH-FILTER} \\
 \frac{G \in \text{dom}(\Omega) \quad \Sigma; G \vdash_{\text{PatLst}} p : \Gamma_p \quad \Sigma; G; \Gamma_p \vdash_1 \phi : \mathbb{B}? \triangleright \mathcal{X} \quad \Sigma; G; \Gamma_p \vdash \mu : \Gamma}{\Sigma \vdash \text{USE } G \text{ MATCH } p \text{ WHERE } \phi \text{ RETURN } \mu : \Gamma} \\
 \text{QRY-MATCH} \\
 \frac{\Sigma \vdash \text{USE } G \text{ MATCH } p \text{ WHERE } \text{TRUE} \text{ RETURN } \mu : \Gamma}{\Sigma \vdash \text{USE } G \text{ MATCH } p \text{ RETURN } \mu : \Gamma}
 \end{array}$$

QRY-MATCH-FILTER checks if the specified graph G is in the catalog Ω , and types the pattern expression p of the **MATCH** clause to obtain its table schema Γ_p , which is used for typing the predicate of the **WHERE** clause as a nullable Boolean in singleton-reference context; and the projection expression of the **RETURN** clause to obtain the result table schema Γ . QRY-MATCH first rewrites with a tautological predicate **TRUE** for **WHERE**, and then delegates to QRY-MATCH-FILTER.

Composite Query Typing. Composite queries compose linear queries under composite operators such as **UNION** and **EXCEPT**, etc., with an optional qualifier (**DISTINCT** v. **ALL**) for choosing set or bag semantics for the operation. Composite queries are covered in ISO §14.2, and its SR:3 (Page 164) states that the composite operators used within a single composite query must all be identical (including the qualifier that decides set v. bag semantics). We design our typing rules to match this.

$$\begin{array}{c}
 \text{COMPQRY-LIFT} \\
 \frac{\Sigma \vdash Q : \Gamma}{\Sigma \vdash_{\otimes} Q : \Gamma} \\
 \text{COMPQRY-COMPOSE} \\
 \frac{\Sigma \vdash_{\otimes} q : \Gamma_q \quad \Sigma \vdash_{\otimes} Q : \Gamma_Q \quad \Gamma_q \sim_{\otimes} \Gamma_Q}{\Sigma \vdash_{\otimes} q \otimes Q : \Gamma_q \otimes \Gamma_Q}
 \end{array}$$

COMPQRY-LIFT lifts a linear query into the composite judgement. COMPQRY-COMPOSE covers query composition by typing the linear query atoms in the context of the composite operator $\otimes$, checking compatibility, and combining their table schemas based on the operator's semantics. Since the judgement carries the composite operator in its context, it automatically enforces the GQL standard's requirement of a single composite query using the same composite operator throughout.

5 Small-Step Operational Semantics

We defined well-formedness of GQL queries in §4 by structuring our typing rules compositionally, i.e., lifting context from query-level down to pattern- and expression-level, and typing them with binding table schemas. We now formally define how these queries are executed. While most prior works have formalized query language semantics denotationally in general, including GQL [11], we use a small-step operational approach. This allows us to explicitly show the complexities of interleaving states during query execution, arising from the non-trivial semantics of GQL.

Since GQL allows queries to specify multiple ways of traversing graphs for pattern matching (ISO §4.11), we first fix the variant that we formalize. Graph pattern matching semantics is configured primarily, and jointly, through two parameters: *path modes* (ISO §4.11.7) and *match modes* (ISO §4.11.9). The path modes impose constraints on matched edge/node frequencies for individual path patterns (P), while the match modes control whether these constraints are enforced across the entire pattern list (p). Since every successful pattern match implies a new record containing bindings to the matched elements, the action of path and match modes can be understood as filtering the binding tables of the composing path patterns and pattern lists, respectively. Path modes filter

Runtime Values ($\omega \in \mathcal{V}$)	Execution Constructs	Runtime Syntax
$\omega ::= c$ <i>constant</i> $ \text{NULL}$ <i>null</i> $ n$ <i>node</i> $ e$ <i>edge</i> $ [\omega, \dots, \omega]$ <i>finite list</i>	$R: \mathcal{A} \rightarrow \mathcal{V}$ <i>record</i> $B: R \rightarrow \mathbb{Z}_{\geq 0}$ <i>bind. table</i> $T: R \times \mathcal{P}(\mathcal{E}) \rightarrow \mathbb{Z}_{\geq 0}$ <i>trail table</i> $P: R \times \mathcal{N}^2 \times \mathcal{P}(\mathcal{E}) \rightarrow \mathbb{Z}_{\geq 0}$ <i>path table</i>	$\hat{P} ::= N \mid \hat{P} E \hat{P} \mid \hat{P} \circ \hat{P} \mid \hat{K} \mid P$ <i>path pattern</i> $\hat{K} ::= [\hat{P}]_Y^K \mid [P]_Y^K \langle P_A, \mathcal{F}, \kappa \rangle \mid P$ <i>quantified path</i> $\hat{p} ::= \hat{P} \mid T \mid \hat{p}, \hat{P} \mid T, T$ <i>pattern list</i> $\hat{Q} ::= Q \mid \langle \mathcal{G}, \hat{p}, \phi, \mu \rangle \mid \langle \mathcal{G}, B, \mu \rangle$ <i>query</i> $\hat{q} ::= \hat{Q} \mid B \mid \hat{q} \otimes \hat{Q} \mid B \otimes B$ <i>comp. query</i>

Lifting and Lowering between Execution Constructs

$$\eta_{\mathcal{G}, E}(B) \triangleq \left\{ \left((R, n_{\text{left}}, n_{\text{right}}, \{e\}) \mid \begin{array}{l} R \in B, e := R(\text{var}(E)), \\ (n_{\text{left}}, n_{\text{right}}) \in \text{ends}_{\mathcal{G}, E}(e) \end{array} \right) \right\}$$

$$\eta_{\mathcal{G}, N}(B) \triangleq \left\{ \left((R, n, n, \emptyset) \mid \begin{array}{l} R \in B, \\ n := R(\text{var}(N)) \end{array} \right) \right\}$$

$$\text{ends}_{\mathcal{G}, E}(e) \triangleq \bigcup_{\substack{\partial \in \text{orn}(E) \\ \Delta(e) = \text{dir}(\partial)}} \begin{cases} \{\Theta(e)\} & \text{if } \partial \rightarrow \\ \{(\pi_2(\Theta(e)), \pi_1(\Theta(e)))\} & \text{if } \partial \leftarrow \\ \{(n_1, n_2), (n_2, n_1) \mid \{n_1, n_2\} = \Theta(e)\} & \text{if } \partial \sim \end{cases}$$

$$\pi_T(P) \triangleq \{ \langle (R, \mathcal{W}) \mid (R, n_{\text{left}}, n_{\text{right}}, \mathcal{W}) \in P \rangle \}$$

$$\pi_B(T) \triangleq \{ R \mid (R, \mathcal{W}) \in T \}$$

Pattern Normalization

$$\text{norm}(p) \triangleq \begin{cases} \text{anon}(N) & \text{if } p = N \\ \text{norm}(P) \text{ anon}(E) \text{ anon}(N) & \text{if } p = P E N \\ \text{norm}(P) \circ [\hat{P}_K]_{\text{vars}(\hat{P}_K)}^K \circ \text{anon}(N) & \text{if } p = P E K N \\ \text{norm}(P) & \text{if } p = (P) \\ [\text{norm}((P))]_{\text{vars}(\text{norm}((P)))}^K & \text{if } p = (P) K \\ \text{norm}(p'), \text{norm}(P) & \text{if } p = p', P \end{cases}$$

$$\text{where } \hat{P}_K \triangleq \text{anon}((\epsilon \epsilon \epsilon)) \text{ anon}(E) \text{ anon}((\epsilon \epsilon \epsilon))$$

Quantifier Upper Bound

$$\text{hi}_{\mathcal{G}}(K) \triangleq \begin{cases} |\mathcal{E}| & \text{if } K \in \{*, +\} \\ 1 & \text{if } K = ? \\ i & \text{if } K = \{i\} \\ j & \text{if } K = \{i, j\} \end{cases}$$

Rename Anonymous Atoms

$$\text{anon}(A) \triangleq \begin{cases} A & \text{if } \text{var}(A) \neq \epsilon \\ A[x_a/\epsilon] & \text{otherwise} \end{cases}$$

$$\text{where } x_a \in \mathcal{A}_{\text{Anon}} \text{ fresh}$$

Fig. 5. Runtime values, execution constructs, runtime syntax, and runtime metafunctions used in our semantics formalization. Runtime terms are shown with a hat (e.g., $\hat{P}$) to differentiate them from the source terms of Figure 2. P_A and $\mathcal{F}$ are the accumulator and the frontier of a quantified path, and κ counts its repetitions.

a path pattern's records, which always represent contiguous paths in a graph, by: (1) TRAIL—graph edges may not be bound more than once; (2) ACYCLIC—graph nodes may not be bound more than once, i.e., matched paths must be acyclic; (3) SIMPLE—similar to ACYCLIC except, graph nodes bound to the head/tail variables of a path pattern may repeat, i.e., matched paths may be cyclic; and (4) WALK—no constraints. Match modes filter a pattern list's records by: (1) DIFFERENT EDGES—graph edges may not be bound more than once; and (2) REPEATABLE ELEMENTS—no additional constraints.

We use the TRAIL path mode and DIFFERENT EDGES match mode in our formalization, as they are the default in popular graph query languages like Cypher, and because they guarantee termination of pattern matching, i.e., graph traversal during pattern matching terminates (regardless of quantifiers) since traversed paths cannot repeat edges and hence are upper-bounded by the graph's edge count.

We start by introducing the runtime values and syntax used in our formalization (§5.1); before moving onto the actual GQL semantics, which similar to our typing rules, is organized into three layers mirroring the calculus in Figure 2: (1) *Value Expressions* (not shown due to space constraints); (2) *Patterns* (§5.2)—graph pattern matching; and (3) *Queries* (§5.3)—the full query evaluation pipeline.

5.1 Runtime Values, Execution Constructs, and Runtime Syntax

Figure 5 summarizes the runtime values, execution constructs, runtime syntax, and metafunctions used in our semantics formalization. The *runtime value domain* $\mathcal{V}$ extends the multi-sorted value

universe $\mathcal{U}$ from §2.1 with property graph elements—nodes (n) and edges (e)—and finite lists. The former ranges over the node/edge sets of all the graphs in a database instance’s catalog Ω , while the latter is constructed at runtime for accumulating bindings across quantified pattern iterations. Lists are also paired with the standard *list concatenation* operation “::” for concatenating two lists.

Definition 5.1 (Records and Binding Tables). A record $R: \mathcal{A} \rightarrow \mathcal{V}$, is a partial function from binding variables to runtime values. Two records are *compatible*: $R_1 \asymp R_2$, when they agree on their common domain; their natural join: $R_1 \bowtie R_2$ is then their union as partial functions, and $\perp$ otherwise. A *binding table* $B: R \rightarrow \mathbb{Z}_{\geq 0}$, is a bag of records R ; their join is the join of all compatible record pairs.

The GQL standard specifies two *execution constructs*: records (R) and binding tables (B), which are used throughout the entire query evaluation pipeline to hold intermediate as well as the final results (Definition 5.1). However, since we consider the TRAIL path mode under the DIFFERENT EDGES matching mode for our pattern matching semantics, we introduce two more execution constructs to simplify formalizing the additional constraints they impose. The first is the *trail table* (T), which simplifies enforcing the DIFFERENT EDGES matching mode constraint across the entire pattern list. Similar to binding tables, trail tables are also bags of records, but each record is also annotated with a set of graph edges ($\mathcal{W}$) bound in that record. So edge-disjointness across trail tables corresponding to different path patterns in a pattern list can be trivially enforced by checking the intersection of the bound-edge sets of their respective records. The second is the *path table* (P), for simplifying enforcing the TRAIL constraint when matching path patterns. Unlike pattern lists, the bindings in a single path pattern match correspond to a single path in the working graph. So the path table builds on top of the trail table entries by annotating them with the graph nodes corresponding to the endpoints of the matched path. This simplifies path composition to a trivial endpoint continuity check, while the same bound-edge sets from the trail tables can be reused for the TRAIL constraint.

Definition 5.2 (Trail Tables). A *trail table entry* is a tuple $(R, \mathcal{W})$ of a record R and the edges $\mathcal{W} \in \mathcal{P}(\mathcal{E})$ bound in that record, i.e., a bound-edge set. A *trail table* T is a bag of trail table entries. We equip trail tables with a commutative monoid $(T, \bowtie, 1_T)$, where $1_T \triangleq \{([], \emptyset)\}$; for implicitly enforcing edge-disjointness when joining them. The *trail product* $\otimes$ combines two trail table entries only when their records join and their bound-edge sets are disjoint, and $\bowtie$ lifts it to trail tables:

$$T_1 \bowtie T_2 \triangleq \left\{ \left\{ t_1 \otimes t_2 \mid \begin{array}{l} t_1 \in T_1, \ t_2 \in T_2, \\ t_1 \otimes t_2 \neq \perp \end{array} \right\} \mid (R_1, \mathcal{W}_1) \otimes (R_2, \mathcal{W}_2) \triangleq \begin{cases} (R_1 \bowtie R_2, \mathcal{W}_1 \cup \mathcal{W}_2) & \text{if } R_1 \bowtie R_2 \neq \perp \text{ and } \\ \perp & \mathcal{W}_1 \cap \mathcal{W}_2 = \emptyset \\ \perp & \text{otherwise} \end{cases} \right\}$$

Definition 5.3 (Path Tables). A *path entry* is a tuple $(R, n_{lft}, n_{rht}, \mathcal{W})$ containing a record, the matched path’s endpoint nodes, and its bound-edges. A *path table* P is a bag of path entries. Path tables compose under $\diamond_{\oplus}$, which implicitly enforces endpoint continuity and edge-disjointness. We parameterize it with the operation $\oplus$ that defines how to combine the records of the two entries:

$$P_1 \diamond_{\oplus} P_2 \triangleq \left\{ \left(R_1 \oplus R_2, n_{lft1}, n_{rht2}, \mathcal{W}_1 \cup \mathcal{W}_2 \right) \mid \begin{array}{l} (R_i, n_{lfti}, n_{rhti}, \mathcal{W}_i) \in P_i \ (i \in \{1, 2\}), \\ R_1 \oplus R_2 \neq \perp, \ n_{rht1} = n_{lft2}, \ \mathcal{W}_1 \cap \mathcal{W}_2 = \emptyset \end{array} \right\}.$$

The parametrization is important because although the semantics of path composition remains unchanged between quantified and unquantified path patterns (it only depends on endpoint continuity and edge-disjointness), how variables may bind to nodes/edges differs. Unquantified variables cannot bind to more than one node/edge per record, while quantified variables may bind to multiple depending on the quantifier used. Parametrization allows us to enforce these differences by defining how records combine during path composition. *Path concatenation* $\diamond \triangleq \diamond_{\bowtie}$ instantiates $\oplus$ with the natural join of records, while Definition 5.4 defines it for quantified path patterns.

Definition 5.4 (Quantified Path Repetition). Let $\mathcal{Y} = \mathbf{vars}(P)$ be the variables declared by a quantified path and let K be its quantifier. The quantifier repetition is bounded below by $\mathbf{lo}(K)$ (Figure 4) and above by $\mathbf{hi}_G(K)$ (Figure 5), which caps the unbounded quantifiers by the number of graph edges (TRAIL constraint of edge-disjointness). Variables declared inside the quantifier accumulate bindings by the *repetition extension* $\diamond_K \triangleq \diamond_{\star_K}$, starting from the *zero-repetition table* $0_{\mathcal{Y}}^K$; for $x \in \mathcal{Y}$:

$$0_{\mathcal{Y}}^K \triangleq \left\{ \left\{ \left[x \mapsto \begin{cases} \mathbf{NULL} & \text{if } K = ? \\ \square & \text{else} \end{cases} \right] \mid n \in \mathcal{N} \right\} \mid n \in \mathcal{N} \right\} \quad (R_1 \star_K R_2)(x) \triangleq \begin{cases} R_2(x) & \text{if } K = ? \\ R_1(x) :: [R_2(x)] & \text{else} \end{cases}$$

The resulting group representation is the runtime counterpart of the quantifier lift $(\cdot) \uparrow_K$ (Definition 4.2). It is list-valued for every quantifier other than $?$, which admits a single repetition and keeps the binding itself on successful match, or **NULL** otherwise. The zero-repetition table is a path table pairing every node in the graph with itself, so joining it with P under $\diamond_K$ leaves endpoints and bound-edge sets unchanged and only lifts each record into its group representation. Therefore, $0_{\mathcal{Y}}^K \diamond_K P$ is the group representation of one repetition, and repeating it $\kappa - 1$ times further, yields the matches that use exactly κ repetitions.

5.2 Pattern Semantics

Pattern matching is responsible for introducing bindings into the query pipeline. A **MATCH** clause enumerates every homomorphism of its pattern into the working graph, which the standard specifies as a multi-phase pipeline (ISO §22.3)—each path pattern is evaluated independently, followed by forcing agreement on shared variables via natural equijoins on the cross products of their binding tables, producing the final result table. We formalize pattern matching using three main judgements.

$$\mathcal{G} \vdash_{\text{Atom}} A \Downarrow B \quad \mathcal{G} \vdash_{\text{Pat}} \hat{P} \rightarrow \hat{P}' \quad \mathcal{G} \vdash_{\text{PatLst}} \hat{p} \rightarrow \hat{p}'$$

The *atom judgement* matches a pattern atom A , i.e., a node N or an edge E atom, in isolation to produce a binding table B of singleton records. It is big-step ($\Downarrow$) as it only depends on the working graph. The *path judgement* reduces path patterns (Figure 5) to path tables with endpoint continuity and edge-disjointness enforced. The *pattern list judgement* reduces pattern lists to trail tables by composing path patterns using conjunction (p, P), while enforcing edge-disjointness throughout.

Atom Matching (Level 1). The first step in atom matching is to determine if the labels of a graph element ι (node or edge) satisfy the label expression l . We use an auxiliary judgement $\iota \vdash_{\Lambda} l \Downarrow b$ for this, which evaluates to a Boolean $b \in \{\mathbf{TRUE}, \mathbf{FALSE}\}$ indicating whether the labels of ι satisfy l .

$\frac{\text{L-EMPTY}}{\iota \vdash_{\Lambda} \varepsilon \Downarrow \mathbf{TRUE}}$	$\frac{\text{L-ATOM} \quad L \in \Lambda(\iota)}{\iota \vdash_{\Lambda} L \Downarrow \mathbf{TRUE}}$	$\frac{\text{L-ATOM-FAIL} \quad L \notin \Lambda(\iota)}{\iota \vdash_{\Lambda} L \Downarrow \mathbf{FALSE}}$	$\frac{\text{L-WILD} \quad \Lambda(\iota) \neq \emptyset}{\iota \vdash_{\Lambda} \% \Downarrow \mathbf{TRUE}}$	$\frac{\text{L-WILD-FAIL} \quad \Lambda(\iota) = \emptyset}{\iota \vdash_{\Lambda} \% \Downarrow \mathbf{FALSE}}$
$\frac{\text{L-NEG} \quad \iota \vdash_{\Lambda} l \Downarrow b}{\iota \vdash_{\Lambda} !l \Downarrow \neg b}$	$\frac{\text{L-AND} \quad \iota \vdash_{\Lambda} l_1 \Downarrow b_1 \quad \iota \vdash_{\Lambda} l_2 \Downarrow b_2}{\iota \vdash_{\Lambda} l_1 \& l_2 \Downarrow b_1 \wedge b_2}$		$\frac{\text{L-OR} \quad \iota \vdash_{\Lambda} l_1 \Downarrow b_1 \quad \iota \vdash_{\Lambda} l_2 \Downarrow b_2}{\iota \vdash_{\Lambda} l_1 l_2 \Downarrow b_1 \vee b_2}$	

The rules follow the same structure as their counterparts in atom typing (§4.2), which filter schemas.

$\frac{\text{ATOM-NODE} \quad B := \left\{ \left[\mathbf{var}(N) \mapsto n \mid \begin{array}{l} n \in \mathcal{N}, \\ \mathbf{prp}(N) \subseteq \Xi(n), \\ n \vdash_{\Lambda} \mathbf{lbl}(N) \Downarrow \mathbf{TRUE} \end{array} \right] \right\}}{\mathcal{G} \vdash_{\text{Atom}} N \Downarrow B}$	$\frac{\text{ATOM-EDGE} \quad B := \left\{ \left[\mathbf{var}(E) \mapsto e \mid \begin{array}{l} e \in \mathcal{E}, \mathbf{prp}(E) \subseteq \Xi(e), \\ e \vdash_{\Lambda} \mathbf{lbl}(E) \Downarrow \mathbf{TRUE}, \\ \Lambda(e) \in \mathbf{dir}(\mathbf{ornt}(E)) \end{array} \right] \right\}}{\mathcal{G} \vdash_{\text{Atom}} E \Downarrow B}$
--	---

ATOM-NODE collects every graph node whose property map is a superset of the atom's property map and whose labels satisfy its label expression, binding each of them to $\mathbf{var}(N)$ in a singleton record. ATOM-EDGE is the symmetric rule for edges, with an additional predicate for directionality.

Path Pattern Matching (Level 2). Path pattern matching (ISO §16.7) reduces a path pattern to a path table (Definition 5.3), leveraging the latter's implicit enforcement of endpoint continuity and edge-disjointness constraints for simplifying the formalization of the former's semantic rules.

$$\begin{array}{c}
\text{PATH-NODE} \\
\frac{\mathcal{G} \vdash_{\text{Atom}} N \Downarrow B}{\mathcal{G} \vdash_{\text{Pat}} N \rightarrow \eta_{\mathcal{G}, N}(B)}
\end{array}
\quad
\begin{array}{c}
\text{PATH-EDGE-L} \\
\frac{\mathcal{G} \vdash_{\text{Pat}} \hat{P} \rightarrow \hat{P}'}{\mathcal{G} \vdash_{\text{Pat}} \hat{P} EN \rightarrow \hat{P}' EN}
\end{array}
\quad
\begin{array}{c}
\text{PATH-EDGE-R} \\
\frac{\mathcal{G} \vdash_{\text{Pat}} N \rightarrow P_2}{\mathcal{G} \vdash_{\text{Pat}} P_1 EN \rightarrow P_1 EP_2}
\end{array}
\quad
\begin{array}{c}
\text{PATH-EDGE} \\
\frac{\mathcal{G} \vdash_{\text{Atom}} E \Downarrow B}{\mathcal{G} \vdash_{\text{Pat}} P_1 EP_2 \rightarrow P_1 \diamond \eta_{\mathcal{G}, E}(B) \diamond P_2}
\end{array}$$

PATH-NODE lifts the binding table (B) with singleton records from a node atom (N) match into a path table with empty bound-edges, i.e., $\mathcal{W} = \emptyset$; using the metafunction $\eta_{\mathcal{G}, N}(B)$ defined in Figure 5. PATH-EDGE-L and PATH-EDGE-R reduce the two sides of $\hat{P} EN$. PATH-EDGE applies once both are path tables by lifting the binding table of the edge atom (E) using the edge-symmetrical metafunction $\eta_{\mathcal{G}, E}(B)$, and then joining the path tables via path concatenation $\diamond$. This is the runtime counterpart of the endpoint condition $\theta_E(\cdot, \cdot, \cdot)$ from §4.2; handling edge orientation via $\mathbf{ends}_{\mathcal{G}, E}(e)$.

$$\begin{array}{c}
\text{PATH-CONCAT-L} \\
\frac{\mathcal{G} \vdash_{\text{Pat}} \hat{P}_1 \rightarrow \hat{P}_1'}{\mathcal{G} \vdash_{\text{Pat}} \hat{P}_1 \circ \hat{P}_2 \rightarrow \hat{P}_1' \circ \hat{P}_2}
\end{array}
\quad
\begin{array}{c}
\text{PATH-CONCAT-R} \\
\frac{\mathcal{G} \vdash_{\text{Pat}} \hat{P}_2 \rightarrow \hat{P}_2'}{\mathcal{G} \vdash_{\text{Pat}} P_1 \circ \hat{P}_2 \rightarrow P_1 \circ \hat{P}_2'}
\end{array}
\quad
\begin{array}{c}
\text{PATH-CONCAT} \\
\frac{P := P_1 \diamond P_2}{\mathcal{G} \vdash_{\text{Pat}} P_1 \circ P_2 \rightarrow P}
\end{array}
\quad
\begin{array}{c}
\text{PATH-QPATH-STEP} \\
\frac{\mathcal{G} \vdash_{\text{Pat}} \hat{P} \rightarrow \hat{P}'}{\mathcal{G} \vdash_{\text{Pat}} [\hat{P}]_Y^K \rightarrow [\hat{P}']_Y^K}
\end{array}
\quad
\begin{array}{c}
\text{PATH-QPATH} \\
\frac{\mathcal{G} \vdash_{\text{QPat}} \hat{K} \rightarrow \hat{K}'}{\mathcal{G} \vdash_{\text{Pat}} \hat{K} \rightarrow \hat{K}'}
\end{array}$$

PATH-CONCAT-L, PATH-CONCAT-R, and PATH-CONCAT reduce path patterns connected by GQL's concatenation operator $\circ$. This operator allows connecting path patterns at their endpoint node atoms by enforcing that the variables corresponding to these endpoint node atoms must agree, i.e., are the same. We do not consider this in the source calculus (Figure 2) but introduce it in the runtime calculus (Figure 5) for conveniently desugaring quantified edges into quantified path patterns with anonymous (variable declaration absent) endpoint node atoms. PATH-QPATH-STEP similarly steps the path inside a quantifier frame $[\cdot]_Y^K$ while preserving the frame itself, and PATH-QPATH delegates it to the auxiliary quantified path judgement once the path pattern is fully reduced to a path table.

GQL's quantified paths are parallel to regular-expression repetition. Their semantics from the standard can be interpreted using our execution constructs, as the union $P^{\mathbf{lo}(K)} \cup \dots \cup P^{\mathbf{hi}_{\mathcal{G}}(K)}$ of the path tables obtained by iterating the inner path table P between the quantifier's bounds. We compute this union with a frontier. A quantified path runtime term $[P]_Y^K \langle P_A, \mathcal{F}, \kappa \rangle$ carries the *accumulator* P_A of the matched bindings, accumulated starting from repetition $\mathbf{lo}(K)$ to repetition $\kappa - 1$; while the *frontier* $\mathcal{F}$ contains only those bindings matched using exactly κ repetitions.

$$\begin{array}{c}
\text{QPATH-INIT} \\
\frac{P_A := \text{if } \mathbf{lo}(K) = 0 \text{ then } 0_Y^K \text{ else } \{\} \quad \mathcal{F} := 0_Y^K \diamond_K P}{\mathcal{G} \vdash_{\text{QPat}} [P]_Y^K \rightarrow [P]_Y^K \langle P_A, \mathcal{F}, 1 \rangle}
\end{array}
\quad
\begin{array}{c}
\text{QPATH-FINISH} \\
\frac{\kappa > \mathbf{hi}_{\mathcal{G}}(K) \vee \mathcal{F} = \{\}}{\mathcal{G} \vdash_{\text{QPat}} [P]_Y^K \langle P_A, \mathcal{F}, \kappa \rangle \rightarrow P_A}
\end{array}$$

$$\begin{array}{c}
\text{QPATH-ITER} \\
\frac{\kappa \leq \mathbf{hi}_{\mathcal{G}}(K) \quad \mathcal{F} \neq \{\} \quad P_{A'} := \text{if } \mathbf{lo}(K) \leq \kappa \text{ then } P_A \uplus \mathcal{F} \text{ else } P_A \quad \mathcal{F}' := \text{if } \kappa < \mathbf{hi}_{\mathcal{G}}(K) \text{ then } \mathcal{F} \diamond_K P \text{ else } \{\}}{\mathcal{G} \vdash_{\text{QPat}} [P]_Y^K \langle P_A, \mathcal{F}, \kappa \rangle \rightarrow [P]_Y^K \langle P_{A'}, \mathcal{F}', \kappa + 1 \rangle}
\end{array}$$

QPATH-INIT initiates the iteration by computing the one-repetition frontier from the zero-repetition table by extending it with the inner path table (Definition 5.4), followed by instantiating the accumulator path table P_A with the zero-repetition table itself iff the quantifier lower bound $\mathbf{lo}(K)$

is zero. This enables supporting GQL's *empty match* feature for quantifiers. QPATH-ITER applies as long as the counter tracking the number of repetitions κ is not greater than the quantifier upper bound $\mathbf{hi}_{\mathcal{G}}(K)$, and as long as the current frontier $\mathcal{F}$ is not empty. In each iteration, it accumulates the current frontier $\mathcal{F}$ into the accumulator P_A (if $\kappa \geq \mathbf{lo}(K)$), and then computes the next frontier $\mathcal{F}'$ by extending the current one with the inner path table P . Since all extensions happen using the repetition extension operator $\diamond_K$, endpoint continuity and edge-disjointness are implicitly enforced.

Pattern List Matching (Level 3). Pattern list matching composes path patterns via conjunction (p, P) , whose cross product followed by equijoin semantics is captured by our trail-monoid's join $\bowtie$ (Definition 5.2). It joins on the shared variables while enforcing DIFFERENT EDGES at the same time.

$$\begin{array}{c}
\text{PATLST-LIFT-STEP} \quad \frac{\mathcal{G} \vdash_{\text{Pat}} \hat{P} \rightarrow \hat{P}'}{\mathcal{G} \vdash_{\text{PatLst}} \hat{P} \rightarrow \hat{P}'} \quad
\text{PATLST-LIFT} \quad \frac{T := \pi_T(P)}{\mathcal{G} \vdash_{\text{PatLst}} P \rightarrow T} \quad
\text{PATLST-CONJ-L} \quad \frac{\mathcal{G} \vdash_{\text{PatLst}} \hat{P} \rightarrow \hat{P}'}{\mathcal{G} \vdash_{\text{PatLst}} \hat{P}, \hat{P} \rightarrow \hat{P}', \hat{P}} \quad
\text{PATLST-CONJ-R} \quad \frac{\mathcal{G} \vdash_{\text{PatLst}} \hat{P} \rightarrow \hat{P}'}{\mathcal{G} \vdash_{\text{PatLst}} T_1, \hat{P} \rightarrow T_1, \hat{P}'} \quad
\text{PATLST-CONJ} \quad \frac{T := T_1 \bowtie T_2}{\mathcal{G} \vdash_{\text{PatLst}} T_1, T_2 \rightarrow T}
\end{array}$$

PATLST-LIFT-STEP lifts a single path pattern step into the pattern list judgement, and PATLST-LIFT lowers the reduced path table of a path pattern into a trail table by projecting only the record and bound-edge set attributes via $\pi_T(\cdot)$ from Figure 5. PATLST-CONJ-L and PATLST-CONJ-R reduce either side of a conjunction left-to-right, while PATLST-CONJ joins their trail tables when fully reduced.

5.3 Query Semantics

Formalizing query evaluation semantics is yet again relatively simple since most of the complexity has been delegated to value expression and pattern matching semantics, with only the formalization of the flow of the binding tables between query clauses remaining. We use two judgement forms:

$$\begin{array}{c}
\Omega \vdash \hat{Q} \rightarrow \hat{Q}' \\
\text{Linear Query}
\end{array}
\quad
\begin{array}{c}
\Omega \vdash_{\otimes} \hat{q} \rightarrow \hat{q}' \\
\text{Composite Query}
\end{array}$$

The *linear query judgement* rewrites linear queries in the context of the database catalog Ω into one of two intermediate tuples: (1) $\langle \mathcal{G}, \hat{p}, \phi, \mu \rangle$ —for formalizing **MATCH** and **WHERE** clause evaluation, and (2) $\langle \mathcal{G}, B, \mu \rangle$ —for formalizing **RETURN** clause evaluation. The *composite query judgement* formalizes composite query evaluation where the semantics depend on the composite operator $\otimes$.

Linear Query Evaluation. Focused linear queries first select a working graph, then match a pattern against it, optionally filter the matched bindings, before projecting to generate the result.

$$\begin{array}{c}
\text{Q-USE} \quad \frac{G \in \mathbf{dom}(\Omega) \quad \mathcal{G} = \Omega(G) \quad \hat{p} := \mathbf{norm}(p)}{\Omega \vdash \mathbf{USE} \ G \ \mathbf{MATCH} \ p \ \mathbf{WHERE} \ \phi \ \mathbf{RETURN} \ \mu \rightarrow \langle \mathcal{G}, \hat{p}, \phi, \mu \rangle} \quad
\text{Q-MATCH} \quad \frac{\mathcal{G} \vdash_{\text{PatLst}} \hat{p} \rightarrow^* T}{\Omega \vdash \langle \mathcal{G}, \hat{p}, \phi, \mu \rangle \rightarrow \langle \mathcal{G}, T, \phi, \mu \rangle} \\
\\
\text{Q-WHERE} \quad \frac{B := \left\{ \left\{ R \in \pi_{\neg \mathcal{A}_{\text{Anon}}}(\pi_B(T)) \mid \mathcal{G}; R \vdash \phi \rightarrow^* \mathbf{TRUE} \right\} \right\}}{\Omega \vdash \langle \mathcal{G}, T, \phi, \mu \rangle \rightarrow \langle \mathcal{G}, B, \mu \rangle} \quad
\text{Q-RETURN} \quad \frac{B' := \left\{ \left\{ [x_i \mapsto \omega_i] \mid R \in B, \forall (\vartheta_i \ \mathbf{As} \ x_i) \in \mu. \right\} \right\}}{\Omega \vdash \langle \mathcal{G}, B, \mu \rangle \rightarrow B'}
\end{array}$$

Q-USE resolves the graph G against the catalog Ω and normalizes the pattern list of the **MATCH** clause using $\mathbf{norm}(\cdot)$ (Figure 5), which expands its quantified edges and names its anonymous pattern atoms, before rewriting the query to an intermediate tuple. Only the rule for queries with a **WHERE** clause is shown since those without can be rewritten into one. Q-MATCH reduces the pattern list to a trail table using the multi-step closure $\rightarrow^*$ of the pattern list judgement. Q-WHERE filters this trail table by lowering it to a binding table via $\pi_B(\cdot)$ from Figure 5, before dropping the variables introduced by $\mathbf{norm}(\cdot)$ using $\pi_{\neg \mathcal{A}_{\text{Anon}}}(\cdot)$ (as they cannot be referenced in any clause);

and finally retaining only the records for which the predicate reduces to **TRUE**. Q-RETURN projects the optionally filtered binding table using the projection list μ to generate the final binding table.

Composite Query Evaluation. Composite queries compose linear queries via composite operators.

$$\begin{array}{c}
\text{CQ-LIFT} \\
\frac{\Omega \vdash \hat{Q} \rightarrow \hat{Q}'}{\Omega \vdash_{\otimes} \hat{Q} \rightarrow \hat{Q}'} \\
\text{CQ-L} \\
\frac{\Omega \vdash_{\otimes} \hat{q} \rightarrow \hat{q}'}{\Omega \vdash_{\otimes} \hat{q} \otimes \hat{Q} \rightarrow \hat{q}' \otimes \hat{Q}} \\
\text{CQ-R} \\
\frac{\Omega \vdash_{\otimes} \hat{Q} \rightarrow \hat{Q}'}{\Omega \vdash_{\otimes} B_1 \otimes \hat{Q} \rightarrow B_1 \otimes \hat{Q}'} \\
\text{CQ-UNION} \\
\frac{}{\Omega \vdash_{\otimes} B_1 \text{ UNION } B_2 \rightarrow B_1 \uplus B_2}
\end{array}$$

CQ-LIFT lifts linear query stepping into composite query judgement. CQ-L and CQ-R reduce the operands left-to-right until both operands are binding tables. CQ-UNION computes bag union preserving multiplicities. Since the judgement is indexed by the composite operator $\otimes$, the congruence rules thread the same operator through both operands, enforcing the GQL standard's requirement that composite queries use a single composite operator across all their linear queries.

6 Type Soundness

We prove *type soundness*, i.e., every well-formed query produces a binding table when evaluated to completion; that conforms to the schema assigned to the query statically by the typing rules. The typing judgments assign types to *source terms*, while the semantics reduce *runtime terms* containing intermediate forms. Auxiliary conformance and configuration-typing relations bridge this gap.

Definition 6.1 (Value Typing and Conformance). A runtime value $\omega \in \mathcal{V}$ inhabits a type τ , written $\omega : \tau$, when the value is a member of the type's semantic domain: scalars inhabit their base type, **NULL** inhabits $?$ and every nullable type $\tau?$, graph elements inhabit their graph-indexed identity types (up to subtyping), and lists inhabit **List** τ when every element does. Value typing is closed under subtyping: $\omega : \tau$ and $\tau <: \tau_2$ imply $\omega : \tau_2$. A record R then conforms to a record schema Γ , written $R \models \Gamma$, when $\text{dom}(R) = \text{dom}(\Gamma)$ and $\forall x \in \text{dom}(\Gamma). R(x) : \tau. \tau <: \Gamma(x)$, and a binding or trail table conforms to Γ when every record in its support does. A path table P conforms to Γ when every entry $(R, n_{\text{left}}, n_{\text{right}}, \mathcal{W})$ has $R \models \Gamma$, $n_{\text{left}}, n_{\text{right}} \in \mathcal{N}$, and $\mathcal{W} \subseteq \mathcal{E}$; we write $\cdot \models \Gamma$ for all of these.

Configuration Typing. A database world $\Sigma = (\Omega, \Upsilon)$ is *well-formed*, written $\models \Sigma$, when every closed graph site's instance conforms to its declared graph schema (Definition 2.3). *Configuration typing*—written $\Sigma; G \vdash_{\text{pcfg}} \hat{p} : \Gamma_{\text{rt}} \Rightarrow \Gamma$ for a pattern configuration and $\Sigma \vdash_{\text{cfg}} \hat{q} : \Gamma$ for a query configuration—closes the source typing rules under the runtime forms of Figure 5, requiring every table a configuration has materialized to conform to the internal schema of the subterm it replaced, and the clauses that remain to compose that schema into Γ .

Theorem 6.1 (Expression Soundness). Suppose $\models \Sigma$, $\mathcal{G} = \Omega(G)$, and $R \models \Gamma$.

- (1) (*Progress*) If $\vartheta \notin \mathcal{V}$, then there exists ϑ' such that $\mathcal{G}; R \vdash \vartheta \rightarrow \vartheta'$.
- (2) (*Soundness*) If $\Sigma; G; \Gamma \vdash_{\diamond}^{\square} \vartheta : \tau \triangleright X$ is derivable for some $\square, \diamond, X$, and $\mathcal{G}; R \vdash \vartheta \longrightarrow^* \omega$ with $\omega \in \mathcal{V}$, then $\omega : \tau_1$ for some $\tau_1 <: \tau$.

PROOF SKETCH. Progress: every computation rule has a complementary null rule, so a non-value either has a reducible operand or contracts. Soundness: by induction on ϑ , splitting the reduction sequence into subterm evaluations along the left-to-right congruence rules. Computation rules return values at the operator's result type, while the null rules—which fire exactly when an operand's dynamic refinement fails at runtime—return **NULL** at type $?$. Both are subtypes of the nullable types the rules assign, which is why the conclusion is up to subtyping. $\square$

Theorem 6.2 (Pattern Configuration Safety). Suppose $\models \Sigma$, $\mathcal{G} = \Omega(G)$, and $\Sigma; G \vdash_{\text{pcfg}} \hat{p} : \Gamma_{\text{rt}} \Rightarrow \Gamma$.

- (1) (*Progress*) Either $\hat{p} = T$, or there exists $\hat{p}'$ such that $\mathcal{G} \vdash_{\text{PatLst}} \hat{p} \rightarrow \hat{p}'$.
- (2) (*Preservation*) If $\mathcal{G} \vdash_{\text{PatLst}} \hat{p} \rightarrow \hat{p}'$, then $\Sigma; G \vdash_{\text{pcfg}} \hat{p}' : \Gamma_{\text{rt}} \Rightarrow \Gamma$.
- (3) (*Terminal conformance*) If $\hat{p} = T$, then $\pi_{\neg \mathcal{A}_{\text{Anon}}}(\pi_B(T)) \models \Gamma$.

PROOF SKETCH. By induction on the configuration-typing derivation. The path cases use atom conformance and closure of the path-table operations under compatible record joins, and the conjunction case trail-join conformance and its schema-compatibility premise; trail and endpoint checks only remove entries. The quantified case uses an inner induction on κ : the zero-repetition table $\mathbf{0}_Y^K$ inhabits the lifted schema, and each application of $\diamond_K$ preserves it since $\star_K$ extends a list-typed (or, for $?$, nullable) binding by one element of the inner schema's type. Terminal conformance then follows since $\pi_B(\cdot)$ and $\pi_{\neg \mathcal{A}_{\text{Anon}}}(\cdot)$ only erase annotations/attributes that Γ_{rt} adds. $\square$

Corollary 6.1 (Pattern Soundness). If $\models \Sigma$, $\mathcal{G} = \Omega(G)$, $\Sigma; G \vdash_{\text{PatLst}} p : \Gamma$, and $\mathcal{G} \vdash_{\text{PatLst}} \text{norm}(p) \xrightarrow{*} T$, then $\pi_{\neg \mathcal{A}_{\text{Anon}}}(\pi_B(T)) \models \Gamma$.

PROOF SKETCH. Normalization typing supplies the internal schema Γ_{rt} ; iterated preservation and terminal conformance from Theorem 6.2 then give the result. $\square$

Theorem 6.3 (Query Type Soundness). If $\models \Sigma$, $\Sigma \vdash Q : \Gamma$, and $\Omega \vdash Q \xrightarrow{*} B$, then $B \models \Gamma$.

PROOF SKETCH. Progress and preservation for configurations, then iteration. *Progress*: well-formedness of Σ lets a source query resolve its graph site; a pattern-stage configuration steps its pattern list, and transitions to a 3-tuple once that list has reduced to a trail table and every predicate to a value—both terminate, since each pattern rule either reduces a subterm to a table or advances a counter capped by $\text{hi}_{\mathcal{G}}(K) \leq |\mathcal{E}|$, and each expression step strictly decreases the number of non-value subterms; a 3-tuple projects (by Theorem 6.1); a final bag is terminal. *Preservation*: graph resolution (Q-USE) produces a well-typed pattern-stage configuration; pattern stepping (Q-MATCH) preserves it by Theorem 6.2; filtering (Q-WHERE) lowers the trail table, erases the generated attributes—unreferenceable, since no source query mentions an attribute in $\mathcal{A}_{\text{Anon}}$ —and selects a sub-bag, so terminal conformance gives a 3-tuple well-typed at Γ_1 ; and projection (Q-RETURN) applies Theorem 6.1 per expression. Iterating yields $\Sigma \vdash_{\text{cfg}} B : \Gamma$, i.e., $B \models \Gamma$. $\square$

Corollary 6.2 (Composite Query Soundness). If $\models \Sigma$, $\Sigma \vdash q : \Gamma$, and $\Omega \vdash q \xrightarrow{*} B$, then $B \models \Gamma$.

PROOF SKETCH. By induction on the composite-query typing derivation. The base case is Theorem 6.3; in the inductive case CQ-LIFT, CQ-L, and CQ-R reduce the operands to bags conforming to their operand schemas, and every composite operator either merges the two bags or selects one of them, so the result conforms to the composite schema that COMPQRY-COMPOSE assigns. $\square$

7 Mechanized Implementation

We implemented a complete mechanization in Lean 4 comprising more than 23,000 lines across 14 modules, following the GQL calculus in Figure 2. Table 1 summarizes the module layout; the core formalization and its soundness proofs (excluding tests and benchmarks) total roughly 21,300 lines.

The sort hierarchy is encoded as inductive types: BaseSort for T_0 , ExtSort for T_1 (including graph-scoped and schema-refined types), and GSort for full sorts τ , while values use a mutual inductive (Value/ValueList) for kernel-derived DecidableEq over nested lists. Subtyping is an inductive proposition with 17 constructors matching the paper's rules; S-UNION-CONGRUENCE is derived as a theorem from primitives due to a Lean kernel restriction on nested inductives. The typing rules of Section 4 are encoded as eleven inductive propositions totaling 57 constructors across expression, predicate, property-constraint, atom, refinement, pattern, pattern-expression, projection, projection-list, query, and single-operator composite typing.

The semantics module provides executable definitions for all reduction rules, including direction-aware endpoint conditions, frontier-based quantified path iteration, and set operations with duplicate elimination. Beyond the executable definitions, the mechanization proves the full soundness development of Section 6 (Metatheory.lean), the small-step layer with progress, preservation, and a proven-equivalent deterministic interpreter behind a user-facing engine flag (SmallStep.lean), and a certified executable type checker (TypeChecker.lean): inferQuery computes a result schema for any query in the fragment, inferQuery_sound shows every accepted query is well typed in the declarative system, and composition with query type soundness guarantees that an accepted query's result table conforms to the inferred schema. All results depend only on the standard Lean axioms (propext, Classical.choice, Quot.sound), with zero sorry and zero user-declared axioms.

The artifact's tests are layered: 276 unit assertions cover each computational unit and rule, 30 worked examples mirror the paper's definitions one by one, including negative cases, and the six expressible queries of the LDBC SNB Interactive v2 workload [22], translated into GQL as Query terms, form an end-to-end integration test: each is type checked by the certified checker, evaluated against golden results, checked to conform to its inferred schema, and executed on both engines with bit-for-bit agreement. Table 2 shows the queries from the SNB workload, evaluated and type checked using MGQL. In total, the build checks 350 native_decide assertions. Supporting other queries requires features outside the considered calculus, primarily those from GQL's relational core, i.e., data transformations; which have already been mechanized for SQL.

Table 1. Modules in MGQL's Lean4 mechanization.

Module	Formalization	LoC
Sorts.lean	Sec. 3 ($\mathcal{T}_0, \mathcal{T}_1, \mathcal{T}_2$)	254
Values.lean	Sec. 2.1 (values, records)	362
Syntax.lean	Fig. 2 (expr., patterns, queries)	291
Schema.lean	Def. 2.1–2.3 (property graphs)	290
RecordSchema.lean	Def. 3.2–3.4, Sec. 4 (record ops)	375
Subtyping.lean	Sec. 3.5 (subtyping)	297
Typing.lean	Sec. 4 (typing judgments)	1,133
Semantics.lean	Sec. 5 (executable semantics)	638
SmallStep.lean	Sec. 5 (step relations)	2,267
Metatheory.lean	Sec. 6 (soundness proofs)	14,057
TypeChecker.lean	Sec. 4 (executable checker)	1,364
Test.lean	Unit tests (276 assertions)	1,252
Examples.lean	Worked examples (12 files)	404
LDBCench.lean	LDBC SNB integration tests	671
Total		23,655

Table 2. Sample LDBC SNB queries type checked and evaluated using MGQL.

Query	Pattern	Features
IS1	1 directed edge	label, prop constraint
IS3	1 undirected edge	undirected direction
IS4	single node	prop constraint
IS5	1 directed edge	label filter
IC2	2-hop + WHERE	conj., WHERE filter
IC8	3-hop chain	conjunction, left dir.

8 Related Work

Graph Query Languages. Early graph query formalisms centered on RDF and SPARQL [18, 21, 23, 24], which operate on triple-based data and require indirect encodings for the richer metadata found in property graphs. Traversal-based languages such as Gremlin [25] offer fine-grained navigational control at the expense of declarative reasoning. In contrast, pattern-matching languages such as Cypher [15], PGQL [30], and G-CORE [2]; provide a more declarative interface. GQL [11, 20] is the first standardized graph query language for property graphs, standardized as ISO/IEC 39075 in 2024. It unifies ideas from its predecessors and incorporates graph schemas as part of the language specification. Yet the standard's entirely informal specification of GQL's semantics, is not conducive to formal reasoning; leading to potential inconsistencies and ambiguities when implementing it, or even in the standard's specification itself. Francis et al. [13] distilled GQL's pattern matching into a Graph Pattern Calculus (GPC) equipped with typing rules and a denotational semantics under set semantics, and Gheerbrant et al. [16] defined Core GQL and Core PGQ as concise formal models focused on understanding its expressivity. Our work is complementary to these efforts. We

provide a small-step operational semantics, a schema-aware type system with a machine-checked soundness proof; all under bag semantics. None of these are covered by GPC or Core GQL.

Language Mechanization. Mechanizing language specifications in proof assistants has offered several cross-domain benefits. Bodin et al. [7] mechanized ECMAScript 5 in Coq, producing both a formal semantics and an extracted reference interpreter. De Santo et al. [10] gave a comprehensive Coq mechanization of JavaScript regular expressions per ECMA-262, uncovering errors in previous formalisms. Seassau et al. [28] formalized a substantial OCaml subset in Rocq. But GQL’s specification has not been mechanized before, making MGQL the first to mechanize it and thus providing the first formal foundation for rigorously reasoning about its semantics.

Formal Semantics of Query Languages. Relational query languages have been formalized extensively. Guagliardo and Libkin [17] gave a formal semantics for a large SQL fragment, while Benzaken and Contejean [6] and Chu et al. [8] provided mechanized Coq treatments of SQL with nulls, aggregations, and bag semantics. For graph query languages, Regular Path Queries and their extensions [5] primarily focus on providing a clean compact theoretical foundation, leaving out several key features of GQL, Cypher, etc., such as properties, schemas, and bag semantics. Formal semantics exist for core Cypher constructs [3, 15], and Ye et al. [31] proposed a gradual-typing calculus for GQL’s path patterns, but none of these are machine-checked. Díaz et al. [12] mechanized GraphQL in Coq, but GraphQL is a tree-shaped API language and does not contain a graph pattern matching fragment. Our work is the first mechanized formalization of GQL in a proof assistant (Lean4).

9 Conclusion

We have presented MGQL, a formal semantics for a substantial fragment of the ISO/IEC 39075 Graph Query Language. Our development provides three interlocking contributions: a type system that exploits graph schemas for static refinement while tracking three-valued nullability and heterogeneous union types; a small-step operational semantics that makes explicit the interplay among Kleene three-valued logic, trail-aware pattern composition, quantified-path iteration, and the clause-by-clause query pipeline; and a layered type soundness argument establishing that well-typed queries produce results conforming to their declared schemas. MGQL provides the first bridge between GQL’s specification and a mechanized implementation.

Acknowledgments

We thank Cheng Ding, Ivan Grigorik, Linghan Zhong, Lara Marinov, and the anonymous reviewers for helpful feedback and discussions. This work was supported in part by the U.S. National Science Foundation (NSF) Nos. CCF-2217696, CCF-2313027, CCF-2403036; and an Amazon Research Award (Fall 2025). Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the NSF or Amazon.

Data Availability Statement

We mechanized our small-step formalization of the semantics of GQL in Lean4. The artifact supporting this paper is available on Zenodo [29]. It also contains a big-step semantics formalization.

References

- [1] Renzo Angles. 2018. The property graph database model. In *AMW*.
- [2] Renzo Angles, Marcelo Arenas, Pablo Barcelo, Peter Boncz, George Fletcher, Claudio Gutierrez, Tobias Lindaaker, Marcus Paradies, Stefan Plantikow, Juan Sequeda, Oskar van Rest, and Hannes Voigt. 2018. G-CORE: A Core for Future Graph Query Languages. In *SIGMOD*. 1421–1432. doi:10.1145/3183713.3190654
- [3] Renzo Angles, Marcelo Arenas, Pablo Barceló, Aidan Hogan, Juan Reutter, and Domagoj Vrgoč. 2017. Foundations of Modern Query Languages for Graph Databases. In *CSUR*. 1–40. doi:10.1145/3104031
- [4] Renzo Angles, Angela Bonifati, Stefania Dumbrava, George Fletcher, Alastair Green, Jan Hidders, Bei Li, Leonid Libkin, Victor Marsault, Wim Martens, et al. 2023. Pg-schema: Schemas for property graphs. *PACMMOD* (2023), 1–25. doi:10.1145/3589778
- [5] Pablo Barceló Baeza. 2013. Querying graph databases. In *PODS*. 175–188. doi:10.1145/2463664.2465216
- [6] Véronique Benzaken and Évelyne Contejean. 2019. A Coq mechanised formal semantics for realistic SQL queries: formally reconciling SQL and bag relational algebra. In *CPP*. 249–261. doi:10.1145/3293880.3294107
- [7] Martin Bodin, Arthur Charguéraud, Daniele Filaretti, Philippa Gardner, Sergio Maffei, Daiva Naudziuniene, Alan Schmitt, and Gareth Smith. 2014. A trusted mechanised JavaScript specification. In *POPL*. 87–100. doi:10.1145/2578855.2535876
- [8] Shumo Chu, Konstantin Weitz, Alvin Cheung, and Dan Suciu. 2017. HoTTSQL: proving query rewrites with univalent SQL semantics. In *PLDI*. 510–524. doi:10.1145/3140587.3062348
- [9] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *CADE*. 625–635. doi:10.1007/978-3-030-79876-5_37
- [10] Noé De Santo, Aurèle Barrière, and Clément Pit-Claudel. 2024. A Coq Mechanization of JavaScript Regular Expression Semantics. *ICFP* (2024), 1003–1031. doi:10.1145/3674666
- [11] Alin Deutsch, Nadime Francis, Alastair Green, Keith Hare, Bei Li, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Wim Martens, Jan Michels, Filip Murlak, Stefan Plantikow, Petra Selmer, Oskar van Rest, Hannes Voigt, Domagoj Vrgoč, Mingxi Wu, and Fred Zemke. 2022. Graph Pattern Matching in GQL and SQL/PGQ. In *SIGMOD*. 2246–2258. doi:10.1145/3514221.3526057
- [12] Tomás Díaz, Federico Olmedo, and Éric Tanter. 2020. A mechanized formalization of GraphQL. In *CPP*. 201–214. doi:10.1145/3372885.3373822
- [13] Nadime Francis, Amélie Gheerbrant, Paolo Guagliardo, Leonid Libkin, Victor Marsault, Wim Martens, Filip Murlak, Liat Peterfreund, Alexandra Rogova, and Domagoj Vrgoč. 2023. GPC: A Pattern Calculus for Property Graphs. In *PODS*. 241–250. doi:10.1145/3584372.3588662
- [14] Nadime Francis, Amélie Gheerbrant, Paolo Guagliardo, Leonid Libkin, Victor Marsault, Wim Martens, Liat Peterfreund, Alexandra Rogova, and Domagoj Vrgoč. 2023. A Researcher’s Digest of GQL. In *ICDT*. doi:10.4230/LIPIcs.ICDT.2023.1
- [15] Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. 2018. Cypher: An evolving query language for property graphs. In *SIGMOD*. 1433–1445. doi:10.1145/3183713.3190657
- [16] Amélie Gheerbrant, Leonid Libkin, Liat Peterfreund, and Alexandra Rogova. 2025. GQL and SQL/PGQ: Theoretical Models and Expressive Power. *Proc. VLDB Endow.* 18, 6 (2025), 1798–1810. doi:10.14778/3725688.3725707
- [17] Paolo Guagliardo and Leonid Libkin. 2017. A formal semantics of SQL queries, its validation, and applications. *Proc. VLDB Endow.* 11, 1 (2017), 27–39. doi:10.14778/3151113.3151116
- [18] Steve Harris, Andy Seaborne, and Eric Prud’hommeaux. 2013. SPARQL 1.1 Query Language. W3C Recommendation.
- [19] ISO/IEC. 2023. ISO/IEC 9075-2:2023: Information technology — Database languages SQL — Part 2: Foundation (SQL/Foundation). International Standard. <https://www.iso.org/standard/76584.html>
- [20] ISO/IEC. 2024. ISO/IEC 39075:2024 — Information technology — Database languages — GQL. Technical Report. International Organization for Standardization.
- [21] Egor V. Kostylev, Juan L. Reutter, Miguel Romero, and Domagoj Vrgoč. 2015. SPARQL with Property Paths. In *ISWC*. 3–18. doi:10.1007/978-3-319-25007-6_1
- [22] Linked Data Benchmark Council (LDBC). 2023. LDBC SNB Interactive v2 Implementations. https://github.com/ldbc/ldbc_snb_interactive_v2_impls.
- [23] Katja Losemann and Wim Martens. 2013. The complexity of regular expressions and property paths in SPARQL. *ACM Trans. Database Syst.* 38, 4 (2013). doi:10.1145/2494529
- [24] Jorge Perez, Marcelo Arenas, and Claudio Gutierrez. 2009. Semantics and complexity of SPARQL. In *TODS*. 1–45. doi:10.1145/1567274.1567278
- [25] Marko A. Rodriguez. 2015. The Gremlin graph traversal machine and language (invited talk). In *DBPL*. 1–10. doi:10.1145/2815072.2815073

- [26] Marko A. Rodriguez and Peter Neubauer. 2010. Constructions from dots and lines. *Bulletin of the American Society for Information Science and Technology* 36, 6 (2010), 35–41. doi:10.1002/bult.2010.1720360610
- [27] Sherif Sakr, Angela Bonifati, Hannes Voigt, Alexandru Iosup, Khaled Ammar, Renzo Angles, Walid Aref, Marcelo Arenas, Maciej Besta, Peter A. Boncz, Khuzaima Daudjee, Emanuele Della Valle, Stefania Dumbrava, Olaf Hartig, Bernhard Haslhofer, Tim Hegeman, Jan Hidders, Katja Hose, Adriana Iamnitchi, Vasiliki Kalavri, Hugo Kapp, Wim Martens, M. Tamer Özsu, Eric Peukert, Stefan Plantikow, Mohamed Ragab, Matei R. Ripeanu, Semih Salihoglu, Christian Schulz, Petra Selmer, Juan F. Sequeda, Joshua Shnavier, Gábor Szárnyas, Riccardo Tommasini, Antonino Tumeo, Alexandru Uta, Ana Lucia Varbanescu, Hsiang-Yun Wu, Nikolay Yakovets, Da Yan, and Eiko Yoneki. 2021. The future is big graphs: a community view on graph processing systems. *Commun. ACM* 64, 9 (2021), 62–71. doi:10.1145/3434642
- [28] Remy Seassau, Irene Yoon, Jean-Marie Madiot, and François Pottier. 2025. Formal Semantics and Program Logics for a Fragment of OCaml. In *ICFP*. 128–159. doi:10.1145/3747509
- [29] Aditya Thimmaiah, Tong-Nong Lin, and Milos Gligoric. 2026. *MGQL: An Executable, Small-Step Semantics of GQL (artifact)*. doi:10.5281/zenodo.21669636
- [30] Oskar van Rest, Sungpack Hong, Jinha Kim, Xuming Meng, and Hassan Chafi. 2016. PGQL: a property graph query language. In *GRADES*. 1–6. doi:10.1145/2960414.2960421
- [31] Wenjia Ye, Matías Toro, Tomás Díaz, Bruno C. d. S. Oliveira, Manuel Rigger, Claudio Gutierrez, and Domagoj Vrgoč. 2025. Flexible and Expressive Typed Path Patterns for GQL. In *OOPSLA*. 302–328. doi:10.1145/3763061

Received 2026-03-18; accepted 2026-06-10